

ÇANKAYA UNIVERSITY FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

CENG 408

Innovative System Design and Development II Project Report

IoT Integrated Accessible Smart Home System

Hasan Emre Usta – 202111031

Kayra Dalçık – 202111023

Ömer Altıntaş – 202111209

Gökay Çetinakdoğan - 202111050

Advisor: Faris Serdar Taşel

TABLE OF CONTENTS

1. Introduction & Objectives
   1.1. Project Overview
   1.2. Objectives
   1.3. Scope
   1.4. Target Hardware
   1.5. Constraints

2. Literature & Technology Review
   2.1. Modern Smart Home Systems and Accessibility Challenges
      2.1.1. Modern Smart Home Systems and Their Limitations
   2.2. Eye-Tracking in Human–Computer Interaction
      2.2.1. The Role of Eye Movements in Interaction
      2.2.2. Blink-Based Interaction Models
      2.2.3. Gaze-Based Selection and Dwell-Time Interaction
      2.2.4. Head Pose, Eye Fatigue, and Real-World Constraints
      2.2.5. Eye Tracking for Smart Home Interaction: Research Gap
   2.3. Assistive Technologies for People with Disabilities
      2.3.1. EEG, EMG, and Biological Signal-Based Assistive Technologies
      2.3.2. Motion, Gesture, and Wearable-Based Assistive Systems
      2.3.3. Voice-Controlled Assistive Technologies and Their Limitations
      2.3.4. Computer Vision-Based Assistive Systems
      2.3.5. Why Eye Tracking Stands Out Among Assistive Technologies
      2.3.6. Research Gap: Assistive Eye-Tracking for Smart Homes
   2.4. Computer Vision Algorithms and Software Frameworks
      2.4.1. Classical Computer Vision Approaches
      2.4.2. Landmark-Based Face and Eye Analysis
      2.4.3. MediaPipe Face Mesh and Iris Tracking
      2.4.4. Deep Learning–Based Gaze Estimation
      2.4.5. Real-World Performance Considerations
   2.5. Face Recognition Systems
   2.6. IoT Communication Protocols and Smart Home Messaging
      2.6.1. MQTT and the Publish–Subscribe Model in IoT Architectures
      2.6.2. HTTP REST: Ubiquity and Accessibility with Latency Limitations
      2.6.3. Zigbee, Z-Wave, and Thread in Low-Power Smart Home Networks
      2.6.4. Matter and the Evolution of Smart Home Interoperability
      2.6.5. WiFi-Based Communication in Embedded IoT Projects
      2.6.6. MQTT + WiFi in Literature: Use Cases, Performance, and Limitations
   2.7. Hardware Platforms and Processing Considerations
      2.7.1. Raspberry Pi as a Vision-Centric Embedded Computing Platform
      2.7.2. ESP32 as a Low-Cost Wireless IoT Control Unit
      2.7.3. Camera Systems for Eye Tracking Applications
      2.7.4. Alternative Embedded AI Platforms: Jetson Nano, Coral TPU, OpenMV
      2.7.5. Power Consumption, Thermal Performance, and Practical Constraints
      2.7.6. Hardware Selection in Literature: General Patterns and Use Cases
   2.8. Architectural Alternatives
      2.8.1. Edge Processing Architectures
      2.8.2. Cloud-Based Architectures
      2.8.3. Hybrid and Distributed Architectures
      2.8.4. Privacy, Latency, and Energy Trade-Offs in Architecture Selection

2.8.5. Architectural Trends in Literature: General Patterns
2.9. General Evaluation

3. System Requirements
3.1. Overview
3.2. Scope
3.3. Target Users
3.4. System Architecture
3.4.1. Image Capture Layer
3.4.2. Eye-Tracking & Gesture Layer
3.4.3. Smart Home Control Layer
3.5. Hardware Requirements
3.6. Software Requirements
3.7. Functional Requirements
3.7.1. Camera Input
3.7.2. Face & Eye Detection
3.7.3. Gaze Direction
3.7.4. Blink Detection
3.7.5. Debug Logging
3.7.6. Optional GUI
3.7.7. Smart Home Interaction
3.7.8. Manual Command Mode
3.8. Non-Functional Requirements
3.9. Assumptions
3.10. Constraints

4. System Architecture
4.1. Architectural Overview
4.2. Hardware Block Architecture
4.3. Layered System Architecture
4.3.1. User Layer
4.3.2. Video Input Layer
4.3.3. Processing Layer
4.3.4. Gesture Detection and Event Generation Layer
4.3.5. Intent Mapping Layer
4.3.6. User Interface Layer
4.3.7. Communication Layer
4.3.8. IoT Device Control Layer
4.4. End-to-End Control Flow
4.5. Architectural Rationale

5. Implementation
5.1. System Overview
5.1.1. Raspberry Pi Vision Module
5.1.2. Backend API
5.1.3. Mobile Application
5.1.4. Smart Device Integration
5.2. Hardware Implementation
5.2.1. Raspberry Pi 5 as Edge Processing Unit
5.2.2. IP Camera Input
5.3. Raspberry Pi Vision Implementation
5.3.1. Main Vision Loop

5.3.2. Eye Gesture Detection
5.3.3. Gaze Detection
5.3.4. Blink Detection
5.3.5. Event Publishing
5.4. Backend API Implementation
5.4.1. Backend Dependencies
5.4.2. Backend Request Models
5.4.3. Supported Vision Events
5.4.4. Supported Actions
5.4.5. Default Event-to-Action Mapping
5.4.6. System State
5.4.7. UI State
5.4.8. Action Execution Logic
5.4.9. Device Controller
5.4.10. Backend API Endpoints
5.4.11. Vision Event Endpoint Flow
5.4.12. Manual Action Execution
5.4.13. Backward Compatibility
5.5. Mobile Application Implementation
5.5.1. Mobile Dependencies
5.5.2. API Client
5.5.3. Mobile API Functions
5.5.4. Mobile Role in Testing
5.6. Smart Bulb Integration
5.6.1. Environment-Based Device Configuration
5.6.2. Light Command Flow
5.6.3. Plug State
5.7. End-to-End Integration Flow
5.8. Implementation Decisions
5.8.1. Event-Based Architecture
5.8.2. Local Vision Processing
5.8.3. Configurable Mapping
5.8.4. Mock Mode and Real Device Mode
5.8.5. Backward-Compatible Endpoint
5.9. Current Limitations
5.10. Future Improvements
5.11. Summary

6. Testing & Validation
6.1. Testing Strategy
6.2. Test Environment
6.2.1. Hardware Environment
6.2.2. Software Environment
6.3. Backend API Testing
6.3.1. Backend Startup Test
6.3.2. Health Endpoint Test
6.3.3. Status Endpoint Test
6.3.4. Supported Vision Events Test
6.3.5. Supported Actions Test
6.3.6. Mapping Retrieval Test
6.3.7. Mapping Update Test
6.3.8. Invalid Mapping Test

6.4. Action Execution Testing
6.4.1. Manual Action Execution Test
6.4.2. UI Navigation Test
6.4.3. Select Action Test
6.4.4. Back Action Test
6.5. Vision Event Processing Testing
6.5.1. Vision Event Endpoint Test
6.5.2. Default Mapping Behavior Test
6.5.3. Updated Mapping Behavior Test
6.5.4. Unsupported Vision Event Test
6.6. Device Control Testing
6.6.1. Mock Mode Test
6.6.2. Real Smart Bulb Configuration Test
6.6.3. Light ON Test
6.6.4. Light OFF Test
6.6.5. Plug Action Prototype Test
6.7. Raspberry Pi Vision Testing
6.7.1. IP Camera Connection Test
6.7.2. Frame Processing Test
6.7.3. Face and Eye Landmark Detection Test
6.7.4. Gaze Event Test
6.7.5. Blink Event Test
6.7.6. Event Payload Test
6.7.7. Pi to Backend Event Sending Test
6.8. Mobile Application Testing
6.8.1. Backend Connection Test
6.8.2. Status Fetch Test
6.8.3. Vision Event Types Fetch Test
6.8.4. Actions Fetch Test
6.8.5. Mapping Fetch and Update Test
6.8.6. Test Vision Event Sending
6.8.7. Manual Action Execution Test
6.9. End-to-End Validation
6.9.1. End-to-End Test Setup
6.9.2. Full Demo Flow Test
6.9.3. Example Successful Scenario
6.9.4. Physical Device Response Validation
6.10. Error Handling and Edge Case Validation
6.10.1. Backend Offline Case
6.10.2. Camera Unavailable Case
6.10.3. Unsupported Vision Event Case
6.10.4. Unsupported Action Case
6.10.5. Smart Bulb Not Configured Case
6.10.6. Smart Bulb Connection Failure Case
6.11. Validation Summary
6.12. Current Testing Limitations
6.13. Future Testing Improvements
6.14. Conclusion

7. Conclusion & Future Work
7.1. Project Summary
7.2. Key Achievements

7.3. Current Limitations
7.4. Future Work
  7.4.1. Enhanced Gesture Recognition
  7.4.2. Expanded Device Ecosystem
  7.4.3. Improved Accessibility Features
  7.4.4. System Infrastructure
  7.4.5. User Experience
7.5. Closing Statement

1. Introduction & Objectives

1.1. Project Overview

This project aims to design and implement an innovative home automation control system that leverages Internet of Things (IoT) technologies and computer vision for hands-free and accessible smart home interaction. This system is designed to provide independence and improved quality of life for individuals with motor disabilities, while also offering an innovative human-computer interaction paradigm for general users. The system enables users to control smart home devices through eye-tracking and gaze-based commands, eliminating the need for traditional physical interfaces such as switches or voice assistants.

1.2. Objectives

The main objectives of this project are:

- Develop a vision-based user interface capable of detecting and interpreting user eye movements, gaze direction, and blinking patterns in real time.
- Integrate this interface with an IoT control hub that manages smart home devices via wireless communication.
- Create WiFi-enabled prototype devices (e.g., lights, fans, temperature controllers) that respond to gaze-based commands.
- Design an accessible, low-cost, and reliable smart home control solution suitable for individuals with limited mobility and elderly users.

1.3. Scope

The scope of this project includes:

- Implementation of a vision processing module on a Raspberry Pi 4 equipped with a camera module for eye-tracking using MediaPipe or similar Python libraries.
- Development of an IoT control hub using an ESP32 microcontroller that communicates with the Raspberry Pi over WiFi.
- Integration of prototype smart devices that can be wirelessly controlled through the ESP32.
- Evaluation of system performance in terms of response time, accuracy, and user accessibility.

1.4. Target Hardware

- Raspberry Pi 5 (with ip camera for image capture and processing)
- ESP32 microcontroller (for IoT control and communication)
- WiFi-enabled prototype devices (e.g., smart light, fan, temperature controller)
- Power supply units and safety enclosures for all modules

1.5. Constraints

The system design must consider the following constraints:

- Power: Each module should operate efficiently under limited power conditions, suitable for continuous home use.
- Cost: Components should be cost-effective to allow for affordable scalability and real-world deployment.
- Safety: Electrical connections and enclosures must comply with basic electrical safety standards to prevent hazards.
- Camera Privacy: Indicator when camera is active; no data recording or transmission
- Testing Protocol: Comprehensive safety testing before live demonstrations

2. Literature & Technology Review

This section presents a comprehensive literature review on smart home automation platforms, eye-tracking–based human–computer interaction, IoT communication protocols, computer vision algorithms, hardware platforms, and facial recognition technologies. The aim is to establish the scientific and technological foundation of the accessible smart home control system to be developed, to identify components that may be expanded in the future, and to highlight the innovative contributions of the project compared to existing systems.

2.1. Modern Smart Home Systems and Accessibility Challenges

Contemporary smart home technologies, human–computer interaction (HCI) research, and studies in the field of accessibility collectively form the multidisciplinary literature that underpins the development of an eye-tracking–based smart home control system. Although this body of work has been extensively explored in both industry and academia, studies that integrate eye tracking with IoT-based device control remain quite limited. Therefore, a thorough review of previous research is essential for revealing the novel aspects of the system. This section evaluates the existing literature across three main axes: smart home platforms, eye-tracking–based HCI research, and assistive technologies.

2.1.1. Modern Smart Home Systems and Their Limitations

Smart home systems have significantly advanced over the past decade, forming a large ecosystem through platforms such as Google Home, Amazon Alexa, Apple HomeKit, and Samsung SmartThings. These systems coordinate lighting, security, HVAC, energy management, and multimedia devices in an automated manner. Through protocols such as Matter, Zigbee, WiFi, Thread, and Z-Wave, devices from different manufacturers can be controlled from a single centralized interface.

However, a commonly emphasized issue in the literature is that despite their technical capabilities, these systems fall short in terms of accessibility. Modern platforms typically rely on two primary interaction paradigms:

- Mobile application–based control
- Voice command–based control

Mobile applications depend on tapping icons or performing touch gestures on a screen, which requires fine motor skills—posing a significant challenge for individuals with neuromotor impairments. Voice-based control, on the other hand, is unreliable for users with speech impairments, neurological disorders, or those living in noisy environments.

Consequently, the literature indicates that while modern smart home systems are technologically robust, they do not adequately accommodate all user groups. One of the most essential forms of control for individuals with disabilities—contactless interaction—is almost entirely absent from these platforms. Notably, no commercial smart home system today provides a built-in mechanism for controlling devices through eye movements. This gap directly reinforces the innovative value of the proposed project.

2.2. Eye-Tracking in Human–Computer Interaction

Eye tracking holds a distinct position within the field of human–computer interaction (HCI) because eye movements are among the few behavioral indicators that directly and rapidly reflect user intent. In the literature, eye tracking is regarded not only as a sensory input modality but also as a fundamental component of the "natural interaction" paradigm. For users with limited motor abilities, eye movements can serve as a powerful and accessible alternative to traditional input methods such as touch, speech, keyboard use, or gesture-based control.

2.2.1. The Role of Eye Movements in Interaction

HCI literature evaluates eye movements through three primary functions: attention, intention, and direct control. The eyes naturally reveal where a person's attention is directed, providing a strong cue for systems attempting to infer user intent. In attention-based models, the system interprets what the user intends to do by analyzing the focus point on the screen. For example, studies in educational technologies often assess whether students are engaged with learning materials based on their gaze behavior.

In intention-based models, the eyes function as a pointing mechanism. Users can make selections or issue commands simply by looking at a target. The most well-known approach within this paradigm is the "gaze-as-input" model, in which eye movements serve as an input channel comparable to a mouse cursor. The literature demonstrates that this model is particularly effective for enabling computer access for individuals with motor impairments.

The third function, direct control, treats eye movements as explicit control signals. For instance, a leftward gaze can trigger a predefined action. This form of control is crucial for individuals who cannot physically access buttons or traditional interfaces. HCI studies emphasize that because eye movements impose minimal cognitive load, gaze-based control constitutes one of the most natural interaction methods available.

2.2.2. Blink-Based Interaction Models

Blinking is considered one of the earliest and most natural user signals in HCI. In blink-based systems, users trigger actions through single or double blinks. Blink-driven communication systems have historically been used for individuals with ALS, spinal cord injuries, or severe communication impairments. The foundational mechanism is relatively straightforward: the system monitors open/closed eye states and maps them to commands.

However, the literature highlights several limitations of blink-only systems. Prolonged use can cause eye fatigue or dryness, and involuntary blinks may lead to false positives, posing safety concerns. Moreover, blink frequency varies significantly across users. For these reasons, modern HCI research recommends using blinking as a supportive signal rather than the primary interaction method. In the proposed project, blinks may similarly be employed as auxiliary commands to reinforce gaze-direction-based control.

2.2.3. Gaze-Based Selection and Dwell-Time Interaction

The most mature gaze-based interaction model in the literature is gaze-based selection, in which gaze direction is used to control a cursor or select items in an interface. This approach is widely applied in screen-based interaction systems. A user looks at an interface element (e.g., a button), and if the gaze remains fixed on the element for a specified duration, a selection is executed. This mechanism is referred to as dwell-time selection.

HCI research demonstrates that dwell-time values are critical for both system accuracy and user satisfaction.

- If dwell time is too short, unintentional glances may be interpreted as selections.
- If it is too long, interaction becomes slow and cumbersome.

Thus, optimizing dwell time is one of the essential design parameters in gaze-controlled systems.

Gaze-based selection is highly suitable for smart home automation settings as well. Users can simply look at a particular object—such as a lamp icon or the image of a physical device—to communicate their intent. This makes gaze-based interaction particularly valuable in situations where physical contact is impossible or undesirable.

2.2.4. Head Pose, Eye Fatigue, and Real-World Constraints

HCI literature emphasizes that eye-tracking performance must be evaluated not only under ideal laboratory conditions but also in real-world environments. Factors such as changes in head orientation, variations in distance from the camera, the use of eyeglasses, lighting conditions, and user fatigue significantly influence system accuracy.

Eye fatigue is particularly problematic during extended use. For this reason, modern interaction design research suggests employing short interaction cycles and providing users with periodic rest opportunities in gaze-based systems. Eye fatigue is especially prominent in applications requiring continuous cursor control. In contrast, smart home environments, which involve momentary interactions, are naturally well-suited for gaze-based control.

Adaptation to head movement has long been a technical challenge; however, modern systems employing mesh-based facial tracking have substantially mitigated this issue. Even so, HCI studies continue to stress the importance of ergonomically positioning the camera relative to the user for optimal performance.

2.2.5. Eye Tracking for Smart Home Interaction: Research Gap

Although the eye-tracking literature is extensive, studies integrating eye tracking directly with IoT-based smart home control remain scarce. Most existing research focuses on:

- cursor control in screen-based interfaces,
- augmentative and alternative communication (AAC) devices,
- wheelchair navigation systems,
- educational and vision-health applications.

In the domain of smart home automation, gaze-based control has not yet become an established paradigm. Therefore, the proposed project:

- targets a combination that is largely missing in the literature (HCI + IoT + Eye Tracking),
- offers a natural interaction modality for individuals with motor impairments,
- introduces a novel interaction layer to smart home ecosystems.

2.3. Assistive Technologies for People with Disabilities

Assistive technologies developed for individuals who are unable to use traditional human–computer interaction methods due to motor impairments, speech disorders, neurological conditions, or physical disabilities have long been a critical research area in the literature. These technologies aim to help users achieve independence in daily life, communicate effectively, and interact with their environment. Accordingly, the assistive technology literature encompasses a broad conceptual foundation spanning both technological innovation and user experience.

Systems designed for individuals with disabilities often integrate sensing technologies, sensor-based solutions, biological signal analysis, and computer vision techniques. However, a common conclusion in the literature is that no single assistive technology is universally ideal for all users. For this reason, eye tracking has gained increasing attention due to its advantages such as low cost, natural interaction, contactless operation, and high accessibility.

This section examines the assistive technology literature through three primary categories: systems based on biological signals, motion- and gesture-based systems, and computer-vision–based contactless interaction systems.

2.3.1. EEG, EMG, and Biological Signal-Based Assistive Technologies

Technologies based on EEG (Electroencephalography) and EMG (Electromyography) are among the most advanced tools developed for enabling communication and device control in individuals who have experienced significant loss of motor function. EEG-based brain–computer interfaces (BCIs) interpret neural activity and translate it into commands. The literature highlights that such systems play a critical role as communication tools for individuals with severe motor neuron diseases such as ALS.

Despite offering high accuracy, EEG-based solutions present several practical limitations. Electrode placement requires expertise, the systems are costly, neural signals are highly sensitive to noise, and users must wear electrodes on the scalp for extended periods, making daily use inconvenient.

EMG-based systems derive commands from muscle activation; however, they are ineffective for users who have entirely lost muscle function. Furthermore, involuntary muscle contractions may generate false-positive commands. Consequently, the literature generally characterizes biological-signal–based technologies as suitable primarily for laboratory environments rather than practical everyday use for a broad user population.

2.3.2. Motion, Gesture, and Wearable-Based Assistive Systems

Motion sensors, accelerometers, wearable embedded sensors, and glove-based devices represent another major group of assistive technologies frequently explored for enabling interaction in individuals with disabilities. These systems rely on gesture recognition or the extraction of commands from bodily movements. For instance, glove-based systems can detect finger flexion to enable robotic arm control or basic computer interactions.

However, the fundamental limitation of such systems is their dependence on motor movement. For individuals with spinal cord injuries or those experiencing significant loss of muscle tone, these devices become difficult or impossible to use. Additionally, wearable technologies may cause discomfort during prolonged use, require frequent calibration, and pose challenges in adapting to daily routines—issues widely reported in the literature.

Therefore, although motion-gesture systems may be effective for individuals with mild impairments, they fail to offer sufficient accessibility for users with more severe motor limitations. This highlights the growing need for contactless, low-effort interaction methods.

2.3.3. Voice-Controlled Assistive Technologies and Their Limitations

Voice recognition systems—such as Google Assistant, Siri, and Alexa—are among the most widely adopted assistive tools in both commercial and academic contexts. Users can control devices simply by issuing voice commands. However, voice control presents significant limitations in accessibility literature.

Individuals with speech disorders often cannot use these systems reliably. Performance decreases substantially in noisy environments, and some neurological conditions disrupt speech rhythm or articulation, leading to recognition errors. Additionally, privacy concerns prevent many users from adopting voice-activated technologies. Therefore, voice-based control is not considered a universally accessible solution in assistive technology research.

2.3.4. Computer Vision-Based Assistive Systems

In recent years, computer-vision–based assistive technologies have gained rapid prominence in the literature. Camera-based systems offer substantial advantages because they can infer user intent without requiring physical contact. Facial expressions, head orientation, eye openness, and pupil movements can all serve as indicators of user intention.

Expression-based systems interpret gestures such as smiling or raising eyebrows as commands; however, these require strong facial muscle control and therefore are not suitable for all users. Head-orientation–based control may be effective for simple robotic or cursor-control tasks, but prolonged use may lead to neck fatigue.

Within this context, eye tracking stands out as the most effective and user-friendly method among computer-vision–based systems. Eye movements are more isolated from involuntary motor activity, faster to execute, and more natural for individuals with limited motor abilities, making them especially suitable for accessible interaction.

2.3.5. Why Eye Tracking Stands Out Among Assistive Technologies

The literature identifies four key characteristics that make eye tracking unique within assistive technology:

- **Natural interaction:** Eye movements require no additional cognitive or physical effort.
- **Contactless operation:** No muscle force or physical interaction is needed; micro-movements suffice.
- **High accessibility:** Effective even for individuals with severe motor impairments.
- **Low cost:** Camera + software solutions are significantly cheaper than EEG/EMG-based systems.

Consequently, eye-tracking technologies are frequently described as "the most practical and sustainable control method" for individuals with physical disabilities.

2.3.6. Research Gap: Assistive Eye-Tracking for Smart Homes

Despite the breadth of assistive technology research, studies integrating eye tracking with IoT-based smart home control remain extremely limited. Most existing research focuses on:

- screen-based cursor interaction,
- augmentative and alternative communication devices,
- wheelchair navigation systems,
- rehabilitation applications.

Within the context of smart home automation, gaze-based control remains a relatively unexplored domain. Therefore, the proposed project:

- addresses an underdeveloped intersection in the literature (HCI + IoT + Eye Tracking),
- offers a highly innovative and accessibility-focused contribution,
- introduces a new gaze-driven control paradigm capable of managing multiple IoT devices.

2.4. Computer Vision Algorithms and Software Frameworks

In the smart home automation system to be developed, the core component of user interaction is the processing of camera-captured images through computer vision techniques and the extraction of meaningful eye-movement signals from these images. Therefore, understanding the historical progression of computer vision research, examining the operational principles of the algorithms employed, and evaluating the behavior of different methods on embedded hardware platforms are critically important for the project. The following subsections provide a broad overview ranging from classical approaches to modern deep-learning–based solutions, along with a detailed discussion of facial recognition technologies that may be integrated into the system in future stages.

2.4.1. Classical Computer Vision Approaches

Among the earliest methods used for eye detection in the field of computer vision are traditional image-processing–based models. One of the most well-known examples is the Haar Cascade classifier family, which is based on the Viola–Jones framework. Haar features employ simple rectangular filters that capture basic intensity variations in the image, and with the help of the AdaBoost algorithm, a cascade of classifiers enables fast detection of eyes or faces. This method was widely used in early devices with limited computational power.

However, Haar Cascade methods have several significant limitations. Because the features used by the algorithm do not adequately represent the complex structure of the human eye, detection accuracy degrades substantially under variable lighting conditions, partially closed eyelids, or changes in face orientation. Furthermore, these models can

only identify the general location of the eyes—they cannot estimate iris position or gaze direction. For this reason, classical techniques are insufficient for today's real-time, interaction-intensive applications.

2.4.2. Landmark-Based Face and Eye Analysis

Following classical approaches, a major milestone in the literature was the development of landmark-based models capable of identifying specific anatomical points on the face. The 68-point facial landmark model implemented in Dlib is one of the most prominent examples. This model extracts critical landmarks corresponding to the eyes, nose, mouth, and jawline, enabling various geometric computations.

For instance, the Eye Aspect Ratio (EAR) metric is widely used to detect whether the eyes are open or closed, making it effective for blink detection or attention analysis. However, because EAR does not provide direct information about the pupil position, gaze direction can only be inferred indirectly from landmark geometry—and such estimations often suffer from high error rates.

Additionally, the performance of landmark-based models on embedded systems is a major concern. These models typically run at around 8–10 FPS on devices such as Raspberry Pi, which can negatively impact user experience in real-time control systems. Thus, while landmark-based approaches hold academic value, they fall short of meeting the responsiveness requirements of the proposed system.

2.4.3. MediaPipe Face Mesh and Iris Tracking

Modern and high-accuracy solutions in eye tracking became possible with the development of deep-learning–based face and iris models. One of the most powerful tools in this domain is MediaPipe Face Mesh, which, despite its computational efficiency, is capable of generating 468 three-dimensional facial landmarks. One of its subcomponents detects the iris center using four high-precision points, enabling direct estimation of gaze direction.

A key factor behind MediaPipe's success is the BlazeFace detector used for face detection, which operates very efficiently on CPUs. The subsequent mesh regression model generates a stable 3D facial structure that remains robust even under head movements. MediaPipe can achieve 12–25 FPS on Raspberry Pi, making it one of the most suitable solutions for real-time gaze tracking on embedded platforms.

Thanks to its architecture, MediaPipe can simultaneously track blinks, eye openness, iris movement, gaze direction, and head pose. For this reason, it stands out as a strong candidate for the core user-interaction module in this project.

2.4.4. Deep Learning–Based Gaze Estimation

The most accurate gaze-tracking methods in the literature are deep-learning–based models that directly estimate gaze direction. Systems such as Gaze360 and iTracker process both eye and face images to predict three-dimensional gaze vectors. These models rely on large convolutional neural networks (CNNs) and require substantial computational resources. Thus, while they can be executed on GPU-enabled platforms such as Jetson Nano, they are unsuitable for real-time operation on CPU-only embedded systems like Raspberry Pi.

Although these models serve as important accuracy benchmarks in gaze-tracking research, limitations related to power consumption, processing load, and latency make them impractical for the current stage of the project. However, with future enhancements to system architecture—such as cloud-based processing or the use of more powerful edge devices—they remain potential candidates for advanced system upgrades.

2.4.5. Real-World Performance Considerations

The performance of eye-tracking algorithms is determined not only by theoretical accuracy but also by adaptability to real-world conditions. Common real-world challenges highlighted in the literature include variable lighting conditions, eyeglass reflections, changes in the distance between the user and the camera, camera-angle constraints, and user fatigue.

Modern models such as MediaPipe incorporate mechanisms such as mesh stabilization, temporal smoothing, and illumination robustness to mitigate these challenges. These capabilities enable MediaPipe to deliver significantly more stable and resilient performance compared to classical approaches.

2.5. Face Recognition Systems

Although facial recognition technologies are not currently utilized within the scope of the project, they may be incorporated into the system in future stages. Therefore, it is important to review the fundamental approaches and algorithms discussed in the literature.

Haar Cascade Face Detection

This is one of the most classical methods used for face detection. However, it is not stable under modern and variable lighting conditions.

Dlib Face Recognition

This method enables user authentication by generating 128-dimensional facial embeddings. It can run on Raspberry Pi, but its performance is relatively slow.

Deep Learning–Based Face Recognition (FaceNet, ArcFace)

These models represent the state-of-the-art in modern facial recognition. Due to their high accuracy, they have become the standard in contemporary face recognition systems. However, they require GPU acceleration and are therefore difficult to deploy on embedded systems for edge computation.

Facial recognition can be used in the future to support features such as user authentication, personalized profile settings, child safety mechanisms, and automatic system login. For this reason, it presents a significant opportunity for expansion in the future work section of the project.

2.6. IoT Communication Protocols and Smart Home Messaging

For smart home systems to operate reliably and seamlessly, not only the performance of hardware components but also the communication protocols connecting these components play a crucial role. In IoT architectures, communication significantly influences system responsiveness, scalability, fault tolerance, and reliability. As a result, the literature includes an extensive body of research analyzing the strengths and weaknesses of various communication protocols in detail. Particularly in real-time interaction scenarios—such as gaze-based control systems—the selection of an appropriate communication protocol is a critical factor that directly shapes the user experience.

Most IoT protocols are optimized for specific requirements; some aim for low power consumption, while others are designed for high bandwidth or low latency. Therefore, the literature does not designate any single protocol as an “ideal solution”—each protocol offers distinct advantages depending on context. The following subsections examine modern IoT communication methods, their role within smart home environments, and common use cases in the literature through a technical and neutral perspective.

2.6.1. MQTT and the Publish–Subscribe Model in IoT Architectures

MQTT is widely recognized in the literature as one of the most commonly used lightweight communication protocols for IoT applications. It is built on a publish–subscribe architecture, wherein devices communicate not directly with each other but through a broker. This model provides scalable and low-latency communication, especially in scenarios where many devices continuously exchange data over the same network.

Academic studies frequently highlight the following characteristics of MQTT:

- extremely small packet size,
- stable performance under unreliable network conditions,
- a natural fit with event-driven systems,
- low computational overhead on embedded devices.

Due to these advantages, MQTT has been widely adopted in sensor networks, home automation systems, environmental monitoring, and real-time control mechanisms.

However, the literature also notes certain limitations: security configurations must be handled carefully, broker dependency may create a single point of failure in rare cases, and QoS settings require proper management in large distributed environments. Thus, while MQTT is powerful and efficient for lightweight applications, it demands careful configuration in high-security industrial settings.

2.6.2. HTTP REST: Ubiquity and Accessibility with Latency Limitations

HTTP REST remains an important option in IoT projects due to its broad compatibility. REST-based systems are platform-independent, easy to implement, and supported across virtually all device ecosystems. Therefore, REST is often considered a strong choice for IoT scenarios in which data transfer is infrequent, periodic, or of low criticality.

However, the request–response nature of HTTP REST introduces substantial latency. Establishing a new connection for each request, the overhead associated with large HTTP headers, and the lack of built-in publish–subscribe functionality make REST less suitable for real-time interaction environments.

As such, academic studies indicate that REST is typically preferred not in high-frequency or instantaneous control mechanisms but rather in applications requiring larger data transfers or infrequent communication.

2.6.3. Zigbee, Z-Wave, and Thread in Low-Power Smart Home Networks

Zigbee, Z-Wave, and Thread are widely used in commercial smart home systems, primarily due to their energy efficiency and robust mesh networking capabilities. Systems such as Philips Hue, IKEA TRÅDFRI, and SmartThings have relied on these protocols for years. Their low energy consumption enables battery-powered sensors to operate for extended periods.

The mesh network architecture allows devices to function as both endpoints and routers, enabling stable communication across large spaces. This makes Zigbee and Thread particularly advantageous in multi-story homes or wide residential areas.

Nevertheless, the literature identifies several limitations. Zigbee and Z-Wave typically require specialized gateways or dongles, which may increase development costs. Their data transmission speeds are relatively low, making them unsuitable for high-frequency or high-bandwidth applications. As a result, while these protocols are ideal for low-power sensor networks, they are less effective in systems requiring high throughput or low latency.

2.6.4. Matter and the Evolution of Smart Home Interoperability

Matter is a next-generation standard developed by Google, Apple, Amazon, and the Connectivity Standards Alliance (CSA), receiving growing attention in the literature. Its primary goals include enhancing cross-device compatibility, ensuring manufacturer independence, and reducing fragmentation within the smart home ecosystem.

When combined with Thread, Matter supports low-power operation, secure communication, and stable mesh networking. However, academic sources note that Matter is still in its maturation phase and may require advanced technical expertise during development. Limited hardware support can also increase prototyping costs.

Thus, while Matter is considered a key technology for the future of smart home environments, it remains in an early adoption stage within research and educational contexts.

2.6.5. WiFi-Based Communication in Embedded IoT Projects

WiFi is one of the most widely used wireless communication methods in academic and prototyping environments. This is primarily due to its high bandwidth, lack of additional hardware requirements, natural compatibility with embedded platforms, and flexibility during development. Devices such as Raspberry Pi and ESP32, which include built-in WiFi support, further reinforce its widespread use.

WiFi provides stable performance on local networks; however, the literature emphasizes that latency fluctuations may occur under heavy network traffic. Additionally, compared to low-power protocols like Zigbee and Thread, WiFi consumes significantly more energy.

Thus, while WiFi is advantageous for rapid prototyping and high-interaction applications, it is less suitable for battery-powered IoT systems with strict power constraints.

2.6.6. MQTT + WiFi in Literature: Use Cases, Performance, and Limitations

Literature surveys indicate that the combination of MQTT and WiFi is widely preferred in projects requiring low latency, fast data transmission, and embedded-platform compatibility. The ease of integrating this combination with low-cost embedded systems such as Raspberry Pi and ESP32 has made it a popular choice in academic research.

Studies highlight several benefits of this architecture:

- millisecond-level latency,
- simple development ecosystem,
- small packet sizes,
- high stability in local networks,
- modular publish–subscribe structure.

However, the literature also acknowledges that other protocols may provide advantages depending on the application. For instance, Zigbee and Thread offer excellent low-power performance for sensor networks, while Matter is beneficial for long-term device interoperability in large systems. REST-based communication is more appropriate for large data transfers and scenarios where real-time interaction is not required.

Thus, while MQTT + WiFi is recognized as a widely used and high-performing solution, IoT research demonstrates that no single protocol is universally optimal. Protocol selection must be guided by application requirements, energy constraints, and the interaction model.

2.7. Hardware Platforms and Processing Considerations

Selecting a hardware platform for smart home systems is not merely an engineering-oriented decision; it requires a multidimensional evaluation that considers the system's operating model, computational capacity, power consumption, real-time performance requirements, and the accessibility needs of the target user group. A gaze-tracking–based control system necessitates the seamless integration of components such as a camera sensor, image-processing unit, wireless communication modules, and IoT device controllers. Therefore, the literature identifies the comparison of hardware platforms—based on processing capability, ecosystem support, and extensibility—as a critical step in system design.

The following subsections provide a comprehensive examination of commonly used components in this field, including embedded computers, microcontrollers, camera sensors, GPU accelerators, and energy–thermal management modules.

2.7.1. Raspberry Pi as a Vision-Centric Embedded Computing Platform

Raspberry Pi is one of the most widely used platforms in computer vision and IoT projects due to its low cost and extensive community support. Its multi-core ARM Cortex-A72 processor, compatibility with the Python ecosystem, and native support for libraries such as OpenCV and MediaPipe make it an ideal prototyping environment for image-processing tasks.

Academic studies highlight several advantages of Raspberry Pi:

- Ability to perform image processing and real-time tasks concurrently
- Platform-independent Python support enabling rapid development
- Tight integration with camera modules
- Low power consumption (5–7 W)
- Broad compatibility with various sensors and peripherals

Raspberry Pi includes a limited GPU capable of running lightweight CPU-optimized deep learning models. While this restricts the execution of large CNN architectures, it provides adequate performance for optimized models such as MediaPipe Face Mesh. The literature reports stable performance between 12–30 FPS for image processing tasks, which is sufficient for gaze-direction–based interaction.

However, constraints such as limited memory, thermal throttling under heavy computational loads, and inadequate processing capability for large-scale deep learning models are frequently discussed limitations. Thus, Raspberry Pi is generally preferred for lightweight to moderate vision workloads.

2.7.2. ESP32 as a Low-Cost Wireless IoT Control Unit

ESP32 is one of the most widely adopted microcontrollers in the IoT ecosystem and is frequently employed in academic literature for wireless sensor networks, smart home devices, and event-driven control systems. With its low power consumption, built-in WiFi/Bluetooth support, and fast GPIO access, it is ideal for real-time device control.

Key advantages highlighted in research include:

- Stable communication over WiFi and strong compatibility with MQTT
- Millisecond-level response time
- Low energy consumption (~0.2–1 W)
- Extensive open-source development support
- Dual-core processor enabling multitasking

Although ESP32 lacks the processing power required for image processing or heavy data analysis, it performs exceptionally well in tasks involving device control, sensor acquisition, and network communication. As a result, many system architectures in the literature adopt hybrid designs such as Pi + ESP32 or Jetson + ESP32.

2.7.3. Camera Systems for Eye Tracking Applications

Camera selection is a critical hardware design decision for eye-tracking systems. Academic research indicates that camera parameters—including resolution, FPS, lens characteristics, and infrared (IR) support—directly affect algorithmic performance.

Common camera types used in gaze-tracking applications include:

RGB Cameras:
Raspberry Pi Camera Modules, USB webcams, and low-latency CSI cameras are frequently used. Under adequate illumination, RGB cameras provide reliable performance for iris tracking.

Infrared (IR) Cameras:
IR cameras deliver more stable results in low-light environments and offer clearer visibility of the pupil under IR illumination, making them suitable for high-accuracy eye-tracking systems. However, they require more complex hardware and are generally more expensive.

High-FPS Cameras:
Cameras operating at 120 FPS or higher allow the tracking of rapid eye movements; however, they significantly

increase the computational load on embedded devices. For this reason, high-FPS cameras are mainly used in academic research, whereas mid-range FPS (e.g., 30+) is sufficient for real-time prototyping.

The literature indicates that RGB cameras exceeding 30 FPS provide adequate performance for low-latency gaze-based interaction, making Raspberry Pi's camera ecosystem a commonly preferred option.

2.7.4. Alternative Embedded AI Platforms: Jetson Nano, Coral TPU, OpenMV

Several alternative embedded AI platforms have been studied in the literature for running more computationally demanding models:

Jetson Nano:

NVIDIA's GPU-accelerated platform can run CNN-based deep learning models with high performance. It supports advanced gaze-estimation models such as Gaze360 and iTracker. However, its relatively high power consumption (10–20 W), cost, and need for active cooling make it less ideal for small-scale prototypes.

Google Coral Edge TPU:

The Edge TPU accelerates TensorFlow Lite models via dedicated hardware. Although highly efficient, its compatibility is limited to specific model architectures, and some MediaPipe components are not supported.

OpenMV Cam:

With low power consumption and MicroPython support, OpenMV is suitable for basic image-processing tasks; however, it lacks the computational capacity required for robust eye-tracking applications.

Despite their strengths, these platforms are often outweighed by the superior cost–performance balance of Raspberry Pi for gaze tracking + microcontroller-based IoT control. Thus, most academic implementations adopt a Raspberry Pi front-end combined with a microcontroller back-end.

2.7.5. Power Consumption, Thermal Performance, and Practical Constraints

Power and thermal considerations are essential in embedded systems, especially for continuous-vision applications such as gaze tracking, where image processing runs constantly.

Key constraints highlighted in the literature include:

- Raspberry Pi can exceed 80°C under heavy image-processing workloads
- Thermal throttling may reduce FPS and degrade performance
- USB cameras add additional power demands
- ESP32 consumes minimal power but requires more energy during WiFi transmission

Therefore, cooling solutions such as fans, heatsinks, and resolution/FPS optimization strategies are widely employed in academic projects.

2.7.6. Hardware Selection in Literature: General Patterns and Use Cases (Neutral)

A review of the literature reveals consistent patterns in hardware selection for eye tracking, IoT control, and accessibility-focused systems:

- Embedded computers (Raspberry Pi, Jetson Nano) for image processing
- Microcontrollers (ESP32, STM32) for device-level control
- Low-latency RGB or IR cameras as sensing components
- WiFi-based communication for rapid prototyping
- GPU accelerators for large deep learning models

- Zigbee/Thread devices for low-power, energy-critical applications

This diversity demonstrates that different hardware platforms offer advantages in different contexts. The literature acknowledges that no single platform is universally optimal; rather, system architecture, performance requirements, and cost considerations collectively shape hardware selection.

2.8. Architectural Alternatives

In smart home automation systems, the architectural structure plays a decisive role in the system's performance, reliability, latency tolerance, and scalability. For a model that requires gaze-tracking–based user interaction, architectural selection becomes even more critical because commands generated from eye movements must be processed, interpreted, and transmitted to IoT devices within milliseconds. Therefore, the literature identifies various architectural approaches depending on where embedded image processing, IoT communication layers, and control mechanisms are executed.

This section examines three primary architectures frequently discussed in the literature for smart home systems—edge processing, cloud-based processing, and hybrid/integrated architectures—evaluated in terms of their technical foundations, advantages, limitations, and use cases.

2.8.1. Edge Processing Architectures

In edge processing architectures, all image processing, decision-making, user interaction analysis, and control logic are executed directly on local devices. This approach is particularly emphasized in the literature for embedded vision applications and systems requiring real-time performance.

2.8.1.1. Low Latency and Real-Time Performance

One of the most important requirements of gaze-based systems is converting gaze-direction changes into IoT device commands within milliseconds. The literature demonstrates that edge-based solutions provide significantly lower latency compared to cloud-based systems because the image is processed locally, eliminating or minimizing network delay. For this reason, edge processing is the most commonly preferred architecture in real-time HCI applications.

2.8.1.2. Privacy and Data Locality

Keeping image data on the local device offers a major advantage in terms of user privacy. For applications involving sensitive biometric data such as eyes, face, and facial expressions, edge processing is widely recommended in academic and ethical research.

2.8.1.3. Limitations and Computational Constraints

Despite its advantages, edge processing is limited by processing power, memory capacity, and thermal constraints. GPU-enabled devices such as Jetson Nano can overcome these limitations, but at a higher cost. Systems operating on Raspberry Pi are limited to lightweight models. Therefore, edge processing is most suitable for optimized and lightweight vision models.

2.8.2. Cloud-Based Architectures

Cloud processing refers to an architecture in which image-processing tasks are offloaded to powerful remote servers. The local device streams camera data and receives processed results from the cloud. This approach is valuable for running large deep-learning models that cannot be executed on embedded hardware.

2.8.2.1. GPU-Accelerated Model Deployment

Cloud architectures allow running heavy models such as iTracker or Gaze360 with high accuracy on GPU-equipped servers. These models are too computationally demanding for embedded devices but operate effectively in the cloud.

2.8.2.2. Scalability and Centralized Control

Cloud-based systems enable centralized control of many devices. This architecture is widely used in large smart-building deployments and intensive sensor networks.

2.8.2.3. Limitations: Latency, Privacy, and Connectivity

The main disadvantages of cloud architectures include:

- network latency,
- dependence on internet connectivity,
- privacy risks associated with transmitting user image data.

The literature reports internet latency between **40–200 ms**, which is unacceptable for real-time gaze-controlled systems. Therefore, cloud-based architectures are generally unsuitable for eye-tracking applications requiring instantaneous feedback.

2.8.3. Hybrid and Distributed Architectures

Hybrid architectures distribute image processing and decision-making tasks between edge devices and cloud servers. This design is popular in research aiming for both low latency and high accuracy.

2.8.3.1. On-Device Preprocessing + Cloud-Level Deep Analysis

In this approach:

- basic tasks (face detection, landmark extraction, low-level processing) run on the device,
- high-accuracy operations (deep gaze estimation) are offloaded to the cloud.

This reduces cloud load and minimizes data transmission.

2.8.3.2. Adaptive Offloading Strategies

The literature describes adaptive systems that dynamically offload tasks based on CPU/GPU load, balancing energy consumption and performance.

2.8.3.3. Increased Complexity and Reliability Concerns

Hybrid architectures increase system complexity because two separate computing environments must work reliably together.
If the edge component fails, cloud processing is disrupted, and vice versa.
Additionally, hybrid systems usually depend on continuous internet access.

2.8.4. Privacy, Latency, and Energy Trade-Offs in Architecture Selection

Architecture selection is evaluated not only by performance but also by:

Privacy:
Biometric data (eye, face) is extremely sensitive.
Edge processing is the preferred solution in academic literature.

Latency:
Gaze-based systems require millisecond-level responsiveness.
Edge architectures provide the lowest latency.

Energy consumption:
Streaming video to the cloud significantly increases power usage.
WiFi video transmission is one of the major constraints in hybrid architectures.

User experience:
Any interaction delay breaks the sense of direct control, degrading usability.
Thus, real-time HCI literature overwhelmingly favors edge processing.

2.8.5. Architectural Trends in Literature: General Patterns

The literature reveals consistent trends across HCI and smart home research:

- If real-time performance is essential → **edge processing**
- If maximum accuracy is needed → **cloud or GPU-accelerated models**
- If energy efficiency is required → **Zigbee/Thread low-power networks**
- If the system spans many devices → **hybrid architectures**
- If privacy is a priority → **local/edge-only processing**

These trends support the conclusion that **each architecture is optimized for specific scenarios**, and no single model is ideal for all systems.
Final architectural decisions must be based on application needs.

2.9. General Evaluation

This literature review has comprehensively examined the multidisciplinary research domain situated at the intersection of smart home automation, human–computer interaction, eye tracking, IoT communication protocols, computer vision algorithms, and accessibility technologies. Existing studies demonstrate that although smart home platforms have matured technically, they still present significant gaps in terms of accessibility. While systems such as Google Home, Alexa, and SmartThings possess extensive device ecosystems, user interaction is predominantly based on voice commands or mobile applications—both of which create substantial usability barriers for individuals with limited motor abilities.

The human–computer interaction literature identifies eye movements as a natural, rapid, and low-effort interaction modality, emphasizing that gaze-based systems offer high accessibility, particularly for users with motor impairments. However, the integration of eye tracking with IoT-based device control has been explored only to a limited extent in the existing research. This reveals a clear need for a new interaction paradigm within smart home systems.

Computer vision research spans a wide range of techniques—from classical methods to deep-learning models—for gaze tracking, and demonstrates that lightweight models such as MediaPipe Face Mesh can achieve real-time performance on embedded devices. The hardware literature similarly indicates that platforms such as Raspberry Pi, ESP32, and IR/RGB cameras are widely used for low-cost prototyping, while cloud and hybrid architectures are considered for scenarios requiring more computationally intensive models. Research on IoT protocols further shows that technologies such as MQTT, Zigbee, Thread, and Matter each provide distinct advantages and limitations depending on the application context.

Overall, the literature suggests that although the technical foundation of smart home automation is well established, there remains a critical gap in accessibility and alternative interaction methods. Transforming eye movements into a natural control signal and integrating this signal within an IoT ecosystem represents a novel contribution both technically and socially. Therefore, the proposed project directly targets the shortcomings identified in current research and offers an innovative solution aimed at enhancing accessibility in smart home systems.

3. System Requirements

3.1. Overview

The IoT Integrated Accessible Smart Home System enables users to control smart home devices through eye-tracking gestures such as looking left/right and short/long blinks. Manual command execution is also supported as an alternative control method.

3.2. Scope

The system includes:

- Video input via an IP camera
  - The system receives a real-time video stream from an IP camera or a smartphone connected over the local network.
- Eye-tracking and gesture recognition
  - The system analyzes eye movements and blink patterns from the video stream to detect user gestures.
- Smart home integrations (e.g., smart bulbs, smart plugs, Smart TVs)
  - Detected gestures are translated into commands that control supported smart home devices.

3.3. Target Users

- Users with limited hand mobility
  - Provides hands-free interaction for users who have difficulty using traditional input devices.
- Users unable to use voice assistants
  - Offers a non-verbal control method for users who cannot or prefer not to use voice commands.
- Anyone needing an alternative human–computer interaction method
  - Can be used by anyone seeking a different or more accessible interaction approach.

3.4. System Architecture

3.4.1. Image Capture Layer

- Reads video stream from an IP camera.
  - Continuously captures video data from a configurable IP camera source.
- Processes frames on Raspberry Pi using OpenCV.
  - Each video frame is processed locally on the Raspberry Pi using OpenCV for efficiency.

3.4.2. Eye-Tracking & Gesture Layer

- Detects face, eyes, and iris using MediaPipe Face Mesh.
  - Uses MediaPipe Face Mesh to accurately locate facial features and eye landmarks.
- Recognizes gaze direction and blink gestures.
  - Analyzes eye position and blink duration to identify user intentions.
- Produces high-level events such as LOOK_LEFT, LOOK_RIGHT, SHORT_BLINK, LONG_BLINK.
  - Converts low-level detections into meaningful system events.

3.4.3. Smart Home Control Layer

- Maps gesture events to navigation or device-control actions.
  - Associates detected gestures with specific user interface or device commands.
- Communicates with smart devices over LAN.
  - Sends control messages to smart devices using local network communication.
- Supports manual commands as a fallback control method.
  - Allows manual input when gesture-based control is unavailable or unsuitable.

3.5. Hardware Requirements

- Raspberry Pi 5 with network connectivity
  - Acts as the main processing unit for video analysis and device control.
- IP camera or smartphone streaming over LAN
  - Serves as the video input source for eye-tracking.
- Smart devices such as:
  - Smart bulbs
  - Smart plugs
  - Smart TV

3.6. Software Requirements

- Raspberry Pi OS
  - Provides a stable and optimized operating system for the Raspberry Pi.
- Python 3 environment
  - Used for implementing all core system components.
- Core libraries:
  - Essential libraries required for system functionality.
  - OpenCV

    - MediaPipe
    - NumPy
- Optional IoT libraries (MQTT, REST/SDK depending on device brand)
    - python-kasa (for TP-Link Kasa smart bulbs/plugs over LAN)

3.7. Functional Requirements

3.7.1. Camera Input

- Reads video from a configurable IP camera stream.
- Displays an error message if the connection fails.

3.7.2. Face & Eye Detection

- Detects a single user's face and extracts eye/iris landmarks.

3.7.3. Gaze Direction

- Classifies gaze into LEFT, RIGHT, CENTER, UP, or DOWN.
- Generates events when a gaze direction remains stable.

3.7.4. Blink Detection

- Recognizes short blinks as quick-action gestures.
- Recognizes long blinks as confirmation gestures.

3.7.5. Debug Logging

- Periodically logs FPS, gaze state, and gesture events.

3.7.6. Optional GUI

- Displays the camera feed with overlay information.
- Supports exit via keyboard.

3.7.7. Smart Home Interaction

- Converts gestures into actions such as navigation or toggling devices.
- Sends device commands via LAN protocols (MQTT or HTTP).
    - For TP-Link Kasa devices, commands can be sent using python-kasa.
- Smart TV control may include channel switching, volume adjustment, or power toggling.

3.7.8. Manual Command Mode

- Allows users to issue commands manually through text input.
- Useful for configuration or accessibility alternatives.

3.8. Non-Functional Requirements

- Provides real-time processing on Raspberry Pi.
- Recovers gracefully from camera disconnections.
- Allows calibration of gesture sensitivity.
- Processes video locally for privacy.
- Minimizes false gesture triggers.

3.9. Assumptions

- Adequate lighting is present for stable eye tracking.
- Only one user appears on camera.
- The user can perform basic gaze and blink gestures.

3.10. Constraints

- Optimized for Raspberry Pi 5 hardware.
- Accuracy may degrade under extreme lighting.
- Tested primarily with mobile IP camera applications.

4. System Architecture

4.1. Architectural Overview

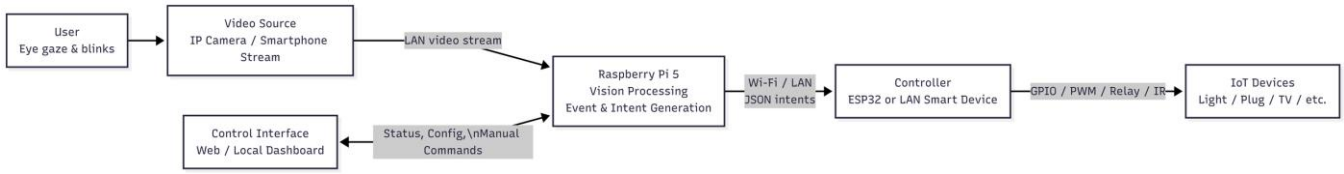
The IoT Integrated Accessible Smart Home System is designed as a layered and modular architecture that enables hands-free interaction with smart home devices using eye-gaze and blink-based gestures. The system integrates real-time computer vision, local decision-making, and network-based IoT control within a single coherent architecture.

At the center of the system is a Raspberry Pi 5, which functions as an edge computing and control hub. It continuously processes a video stream obtained from a camera source, detects gaze and blink gestures, converts these gestures into high-level control intents, and distributes these intents over the local network to device controllers or smart devices. A control interface (UI) is included in the architecture to allow visual feedback, configuration, and gesture-based navigation.

The architecture explicitly separates visual perception, intent generation, user interface logic, and device actuation, which improves scalability, maintainability, and system clarity.

4.2. Hardware Block Architecture

The following block diagram illustrates the main hardware components of the system and the high-level data and control flow between them.



Key architectural notes:

- The camera is treated as a generic video source, not tied to a specific hardware interface.
- The Raspberry Pi focuses on perception and decision-making.
- Physical device control is delegated to dedicated controllers (e.g., ESP32), ensuring electrical safety and modularity.
- The UI communicates with the same intent pipeline used by gaze-based control.

4.3. Layered System Architecture

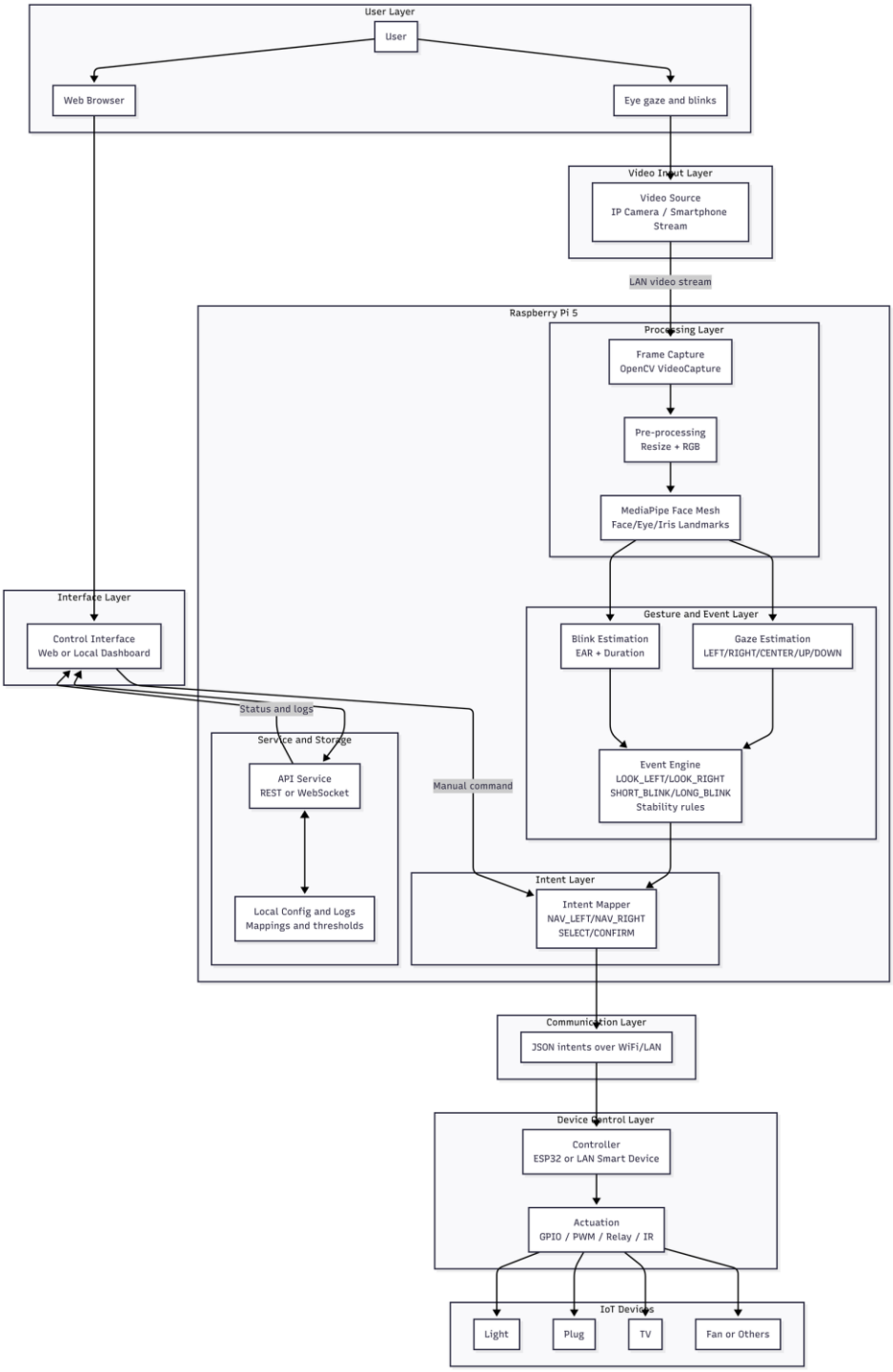


Figure 2. Layered system architecture showing how user input is captured, processed on the Raspberry Pi, translated into gesture events and high-level intents, and delivered to IoT device controllers.

4.3.1. User Layer

The User Layer represents the interaction point between the human user and the system.

Users interact primarily through eye-gaze direction and blink gestures, which serve as non-verbal control inputs. This interaction model is especially suitable for users with limited hand mobility or those who require alternative interaction methods.

The system also provides visual feedback to the user through the control interface and the physical response of the controlled devices.

4.3.2. Video Input Layer

The Video Input Layer is responsible for acquiring real-time visual data.

- Video frames are obtained from a configurable IP camera or smartphone camera stream over the local network.
- Frames are captured on the Raspberry Pi using OpenCV-based video capture and resized to a fixed resolution for stable processing.
- The architecture abstracts the camera as a video source, allowing different camera types to be used without changing the rest of the system.

If the video stream becomes unavailable, the system detects the failure and handles it according to predefined error-handling policies.

4.3.3. Processing Layer (Raspberry Pi 5)

The Processing Layer performs all computationally intensive tasks required for gesture recognition and control.

Key responsibilities include:

- Preprocessing of incoming frames (resizing, color space conversion),
- Extraction of face, eye, and iris landmarks using MediaPipe Face Mesh,
- Runtime performance monitoring (e.g., FPS measurements and logs).

All processing is executed locally on the Raspberry Pi to minimize latency, preserve user privacy, and avoid dependency on external services.

4.3.4. Gesture Detection and Event Generation Layer

This layer converts raw visual landmarks into meaningful interaction events.

Eye-Gaze Detection

- The relative position of the iris within the eye region is used to classify gaze direction as LEFT, RIGHT, CENTER, UP, or DOWN.
- A center tolerance region is applied to suppress minor, unintentional eye movements.

Blink Detection

- Eye Aspect Ratio (EAR) is used to determine eye-open and eye-closed states.
- Blink duration is analyzed to distinguish between short blinks and long blinks.

Stability Mechanisms

- Gaze-based events are emitted only if the detected gaze direction remains stable for a predefined duration.
- These mechanisms significantly reduce false positives and unintended commands.

The output of this layer consists of discrete events such as LOOK_LEFT, LOOK_RIGHT, SHORT_BLINK, and LONG_BLINK.

4.3.5. Intent Mapping Layer

The Intent Mapping Layer translates detected events into device-agnostic control intents.
Rather than binding gestures directly to specific devices, the system generates abstract intents such as:

- NAV_LEFT, NAV_RIGHT
- SELECT
- CONFIRM or TOGGLE

This abstraction allows the same gesture set to be reused across different devices and user interfaces and enables flexible system expansion.

4.3.6. User Interface Layer

The User Interface Layer provides visual feedback and interaction management.
The interface is designed to:

- Display system status, detected gestures, and active intents,
- Present navigable UI elements controlled using gaze and blinks,
- Allow configuration and calibration of system parameters,
- Support manual command input as a fallback interaction method.

The UI consumes the same intents generated by the system, ensuring consistency between gesture-based control and manual interaction.

4.3.7. Communication Layer

The Communication Layer handles the transfer of intents and status information over the local network.

- Some device integrations may use device-specific local protocols via Python libraries (e.g., python-kasa for TP-Link Kasa).
- Control messages are encoded as JSON.
- Communication occurs over Wi-Fi or Ethernet (LAN).
- The architecture is transport-agnostic.

4.3.8. IoT Device Control Layer

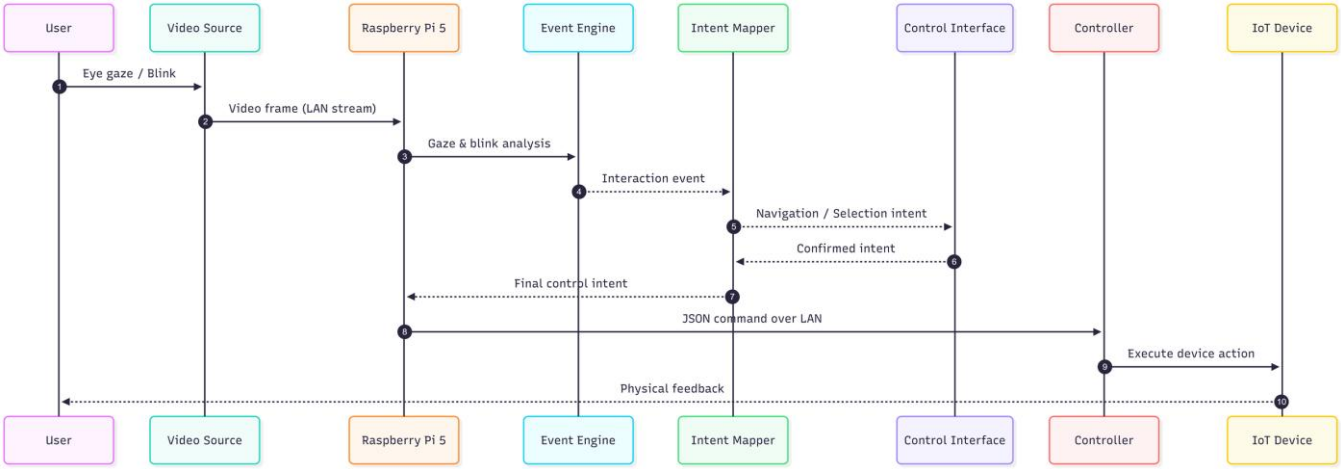
The IoT Device Control Layer executes control intents on physical devices.

- ESP32-based controllers or LAN-enabled smart devices receive intents from the Raspberry Pi.
- Controllers translate intents into device-specific actions using GPIO, PWM, relay switching, or infrared transmission.
- This design avoids direct high-voltage switching from the Raspberry Pi.
- For some LAN-enabled devices (e.g., TP-Link Kasa smart bulbs/plugs), the Raspberry Pi can control devices directly using python-kasa, without requiring an ESP32 controller.

Supported device categories include lighting systems, smart plugs, televisions, and other household appliances.

4.4. End-to-End Control Flow

Figure 3: System Sequence Diagram.



This diagram illustrates that the process starts with the user gazing or blinking at the video source (camera); the captured image data is then transmitted to the Raspberry Pi 5 for processing. The Raspberry Pi analyzes this data ('Gaze & blink analysis') and identifies the user's interaction intent (navigation or selection) via the Event Engine and Intent Mapper modules. Once validated by the Control Interface, this intent is converted into a final control command and sent via the Raspberry Pi to the Controller unit in JSON format. In the final stage, the Controller processes this command to execute the physical action on the target IoT Device, completing the loop by providing physical feedback (such as a light turning on or a sound) to the user indicating that the operation is complete.

4.5. Architectural Rationale

This architecture:

- Clearly separates perception, decision-making, interface logic, and actuation,
- Supports both gesture-based and interface-based control paths,
- Is scalable to multiple devices and controllers,
- Aligns with real-time constraints and privacy considerations,
- Is suitable for incremental implementation across project phases.

5. Implementation

This section describes the engineering implementation of the **IoT Integrated Accessible Smart Home System**. The project was implemented as a modular, event-driven smart home control platform that allows users to control smart home devices through eye movements and blink-based interaction.

The final working system focuses on the following end-to-end flow:

IP Camera → Raspberry Pi Vision Module → Vision Event → FastAPI Backend → Event-to-Action Mapping → Action Execution → Smart Bulb Control

The main implementation principle is that the vision module does **not** directly control smart home devices. Instead, it only detects user interaction and generates standardized vision events. The backend receives these events, maps them to configurable actions, updates the UI/device state, and executes smart device commands when applicable.

This design makes the system modular, testable, and easier to extend with new devices or new interaction methods in future iterations.

5.1. System Overview

The implemented system consists of three main software modules and one real smart device integration:

1. **Raspberry Pi Vision Module**
2. **Backend API**
3. **Mobile Application**
4. **TP-Link/Tapo Smart Bulb Integration**

Each module has a separate responsibility in the system.

5.1.1. Raspberry Pi Vision Module

The Raspberry Pi 5 works as the local vision-processing unit. It receives a live video stream from an IP camera or smartphone camera stream and processes the frames locally using OpenCV and MediaPipe Face Mesh.

The Pi module detects eye-based user interactions and converts them into standardized vision events.

The currently implemented vision events are:

Event	Description
look_left	User looks left for a stable duration
look_right	User looks right for a stable duration
short_blink	User performs a short blink
long_blink	User performs a long blink

The Raspberry Pi sends these events to the backend API. It does not directly execute smart home commands.

5.1.2. Backend API

The backend was implemented using **FastAPI**. It is the central coordination layer of the system.
The backend is responsible for:

- Receiving vision events from the Raspberry Pi
- Receiving test events from the mobile application
- Storing event-to-action mappings
- Executing mapped actions
- Managing UI navigation state
- Managing device state
- Executing smart bulb commands
- Returning system status to the mobile application

The backend follows this core logic:
Vision Event → Mapping Lookup → Action Execution → UI State / Device State Update → Optional Hardware Control

This means that the meaning of each eye gesture can be changed from the mapping layer without changing the Raspberry Pi vision code.

5.1.3. Mobile Application

The mobile application was implemented using **React Native with Expo**. It provides a user interface for control, monitoring, testing, and configuration.
The mobile application communicates with the backend through HTTP requests. It can:

- Fetch backend status
- Fetch supported vision event types
- Fetch supported actions
- Fetch current mappings
- Update event-to-action mappings
- Send test vision events
- Manually execute actions

This makes the mobile app useful both as a user interface and as a development/debugging tool.

5.1.4. Smart Device Integration

For the successful end-to-end demo, the system was integrated with a **TP-Link/Tapo smart bulb** over the local network.

The backend uses a SmartHomeDeviceController class for optional real-device control. If SMART_BULB_HOST is configured in the backend environment, LIGHT_ON and LIGHT_OFF commands are sent to the smart bulb using python-kasa.

If the smart bulb environment variable is not configured, the controller runs in mock mode. This allows the backend, mobile application, and Raspberry Pi pipeline to be tested without requiring physical hardware.

5.2. Hardware Implementation

The prototype was implemented using the following hardware components:

Component	Purpose
Raspberry Pi 5	Local vision processing and event generation
IP camera / smartphone camera stream	Video input source
Local Wi-Fi network	Communication between Pi, backend, mobile app, and smart bulb
TP-Link/Tapo smart bulb	Real smart home device used in the end-to-end demo
Mobile phone	Runs the React Native application
Development computer	Used for backend and mobile development/testing

5.2.1. Raspberry Pi 5 as Edge Processing Unit

The Raspberry Pi 5 was selected as the main edge-processing device because it can run real-time computer vision tasks while remaining affordable and suitable for local deployment.

The Pi processes the video stream locally. This decision provides two major advantages:

1. **Low latency:** Eye-based control should feel responsive. Local processing avoids the delay of sending camera frames to a remote server.
2. **Privacy:** Camera frames are processed locally. The system does not need to upload face or eye images to a cloud service.

5.2.2. IP Camera Input

Instead of depending on a fixed Raspberry Pi camera module, the implementation uses a configurable IP camera stream.

The camera URL is read from environment configuration:

IP_CAMERA_URL

If no custom value is provided, the Pi module uses a default local IP camera stream URL.
The frame size is also configurable through:
FRAME_WIDTH
FRAME_HEIGHT

This makes the camera layer flexible. The video source can be a smartphone IP camera application, a DroidCam-like stream, a USB camera stream, or another network camera source.

5.3. Raspberry Pi Vision Implementation

The Raspberry Pi vision module was implemented in Python.
The main libraries used are:

Library	Purpose
OpenCV	Reading and processing camera frames
MediaPipe	Face Mesh and iris/eye landmark detection
NumPy	Numerical calculations
python-dotenv	Loading environment variables

The Raspberry Pi requirements specify Python 3.11.x and the following core dependencies:
mediapipe==0.10.18
numpy==1.26.4
opencv-python==4.11.0.86
python-dotenv

5.3.1. Main Vision Loop

The Pi module starts by opening the configured IP camera stream. If the camera cannot be opened, the program prints an error message and exits safely.
The main runtime flow is:
Open IP camera
Create MediaPipe Face Mesh instance
Read frame
Convert frame from BGR to RGB
Process frame with Face Mesh

Extract face landmarks
Run gesture detector
Generate vision event if detected
Publish event payload
Send event to backend
Repeat

The Pi uses MediaPipe Face Mesh with the following important settings:

- max_num_faces=1
- refine_landmarks=True
- min_detection_confidence=0.5
- min_tracking_confidence=0.5

Only one face is processed at a time, which matches the current assumption of a single active user interacting with the system.

5.3.2. Eye Gesture Detection

The GestureDetector processes face landmarks produced by MediaPipe Face Mesh.
It uses:

- Eye Aspect Ratio calculations for blink detection
- Iris relative position for gaze direction estimation
- Stability timing for left/right gaze events

The detector produces four main events:
look_left
look_right
short_blink
long_blink

5.3.3. Gaze Detection

The system calculates the iris position relative to the eye region. Based on this relative iris position, the gaze direction is classified.
In the current implementation, stable left and right gaze directions are converted into backend events:

Gaze Direction	Generated Event
LEFT	look_left
RIGHT	look_right

To avoid accidental triggers, the system does not immediately fire an event when the gaze briefly changes. Instead, the gaze direction must remain stable for a configured duration.

The implemented stable gaze duration is:

gaze_stable_time = 0.35 seconds

This reduces false positives caused by short eye movements or noisy landmark detection.

5.3.4. Blink Detection

Blink detection is based on Eye Aspect Ratio (EAR).

The detector uses separate thresholds and duration ranges for short and long blinks.

Important blink parameters include:

ear_threshold = 0.21

long_blink_ear = 0.18

short_blink_min = 0.20 seconds

short_blink_max = 0.35 seconds

long_blink_min = 0.55 seconds

long_blink_max = 1.5 seconds

The generated blink events are:

Blink Type	Event
Short blink	short_blink
Long blink	long_blink

A short blink is mainly used for selecting or confirming an item. A long blink is mainly used for going back or canceling, depending on the current mapping.

5.3.5. Event Publishing

When the Pi detects an eye event, it converts the event into a standardized payload.

Example payload:

```
{
  "source": "eye",
  "name": "look_right",
  "confidence": 1.0,
  "timestamp": 1234567890,
  "metadata": {
    "gaze": "RIGHT",
    "ear": 0.25
```

```
  }
}
```

The event publisher prints the event for debugging and returns a dictionary payload. The integration client then sends this payload to the backend event URL.

The backend event URL is configurable through:

BACKEND_EVENT_URL

The default backend event URL is:

http://127.0.0.1:8000/vision-events

5.4. Backend API Implementation

The backend was implemented as a modular FastAPI service.

The backend title is:

Accessible Smart Home Modular API

Its main design is:

Vision events → user-defined mappings → action execution → smart device control

5.4.1. Backend Dependencies

The backend uses the following dependencies:

fastapi

uvicorn

pydantic

python-kasa

python-dotenv

python-dotenv is used to load environment variables from the backend .env file. python-kasa is used for optional real TP-Link/Tapo smart bulb control.

5.4.2. Backend Request Models

The backend defines three main request models.

VisionEventRequest

Used when the Raspberry Pi or mobile app sends a vision event.

```
{
  "source": "eye",
  "name": "look_left",
```

```
  "confidence": 1.0,
  "timestamp": 1234567890,
  "metadata": {}
}
```

MappingUpdateRequest

Used when the mobile app updates event-to-action mappings.

```
{
  "mappings": {
    "look_left": "NAV_LEFT",
    "look_right": "NAV_RIGHT",
    "short_blink": "SELECT",
    "long_blink": "BACK"
  }
}
```

ActionExecuteRequest

Used when the mobile app manually executes an action.

```
{
  "action": "LIGHT_ON"
}
```

5.4.3. Supported Vision Events

The backend currently supports four vision events:

Event	Description
look_left	User looks left
look_right	User looks right
short_blink	User performs a short blink
long_blink	User performs a long blink

If an unsupported event is sent to the backend, the backend returns a 400 Bad Request response.

5.4.4. Supported Actions

The backend supports both UI navigation actions and device-related actions.

Action	Type	Description
NAV_LEFT	UI action	Moves selected item to the previous item
NAV_RIGHT	UI action	Moves selected item to the next item
SELECT	UI action	Selects the currently highlighted item
BACK	UI action	Returns to the main menu
LIGHT_ON	Device action	Turns the smart bulb on
LIGHT_OFF	Device action	Turns the smart bulb off
PLUG_ON	Prototype device action	Updates plug state to on
PLUG_OFF	Prototype device action	Updates plug state to off

At the current stage, real physical hardware execution is implemented for LIGHT_ON and LIGHT_OFF. Plug actions are represented in backend state and action flow, but they do not yet have a real hardware binding in the device controller.

5.4.5. Default Event-to-Action Mapping

The backend contains a user-editable mapping table.
The default mapping is:

Vision Event	Default Action
look_left	NAV_LEFT
look_right	NAV_RIGHT
short_blink	SELECT
long_blink	BACK

This mapping allows the system to separate gesture detection from action execution.
For example:
look_right → NAV_RIGHT

short_blink → SELECT

If needed, the mobile application can update the mapping. For example, short_blink can be mapped directly to LIGHT_ON for testing or accessibility customization.

5.4.6. System State

The backend keeps an in-memory system_state.
It stores:

- Last received vision event
- Last executed action
- Last executed device command
- Current light status
- Current plug status

The initial device state is:

```
{
  "light": "off",
  "plug": "off"
}
```

This state is returned to the mobile app through the /status endpoint.

5.4.7. UI State

The backend also keeps an in-memory ui_state.
The UI state contains:

- Current screen
- Selected index
- Available screens
- Items inside each screen

The implemented screens are:

Screen	Items
main_menu	Light, Plug, Status
light_menu	On, Off, Back
plug_menu	On, Off, Back

status_menu	Refresh, Back

This allows eye gestures to navigate a simple menu structure.
Example navigation flow:
look_right → NAV_RIGHT → selected item changes
short_blink → SELECT → selected item opens or executes
long_blink → BACK → returns to main menu

5.4.8. Action Execution Logic

The backend normalizes all action names to uppercase and checks whether the action is supported.
NAV_LEFT

Moves the selected index to the previous item. If the selected item is already the first item, it wraps around to the last item.
NAV_RIGHT

Moves the selected index to the next item. If the selected item is already the last item, it wraps around to the first item.
SELECT

Executes behavior based on the current screen and selected item.
Examples:

Current Screen	Selected Item	Result
main_menu	Light	Opens light_menu
main_menu	Plug	Opens plug_menu
main_menu	Status	Opens status_menu
light_menu	On	Executes LIGHT_ON
light_menu	Off	Executes LIGHT_OFF
light_menu	Back	Returns to main menu
plug_menu	On	Executes PLUG_ON
plug_menu	Off	Executes PLUG_OFF
plug_menu	Back	Returns to main menu

status_menu	Refresh	Returns current device status
status_menu	Back	Returns to main menu

BACK

Returns to the main menu unless the user is already on the main menu.

Device Actions

Device actions update the backend device state and call the device controller.

5.4.9. Device Controller

The backend includes a SmartHomeDeviceController class.

This controller has two operating modes:

Mode	Description
mock	Used when SMART_BULB_HOST is not configured
python-kasa	Used when a real TP-Link/Tapo smart bulb is configured
misconfigured	Used when SMART_BULB_HOST exists but python-kasa cannot be imported

The controller reads these environment variables:

SMART_BULB_HOST

KASA_USERNAME

KASA_PASSWORD

If SMART_BULB_HOST is configured, the backend tries to discover and control the TP-Link/Tapo device over the local network.

For real bulb control:

- LIGHT_ON calls device.turn_on()
- LIGHT_OFF calls device.turn_off()

After execution, the device is updated and the backend returns hardware result information such as:

- Controller mode

- Whether the command was applied
- Device host
- Device alias
- On/off status

If the command is not LIGHT_ON or LIGHT_OFF, the controller returns a mock result explaining that there is no real device binding for that command.

5.4.10. Backend API Endpoints

The final backend exposes the following endpoints:

Endpoint	Method	Purpose
/	GET	Returns a basic API running message
/health	GET	Returns backend health, device controller mode, and UI snapshot
/status	GET	Returns system state, UI state, controller mode, and mappings
/vision-event-types	GET	Returns supported vision event names
/actions	GET	Returns supported actions
/mappings	GET	Returns current mappings and supported event/action lists
/mappings	PUT	Updates event-to-action mappings
/actions/execute	POST	Manually executes an action
/vision-events	POST	Receives a vision event and executes the mapped action
/event	POST	Backward-compatible alias for /vision-events

5.4.11. Vision Event Endpoint Flow

When a request is sent to /vision-events, the backend performs these steps:

1. Normalizes the event name.
2. Checks whether the event is supported.
3. Saves the event as last_vision_event.
4. Looks up the mapped action from mapping_store.
5. Executes the mapped action.

6. Updates UI state or device state.
7. Returns the mapped action, action result, UI state, and device status.

Example:
look_right → NAV_RIGHT → selected menu item changes

Another example:
short_blink → SELECT → selected menu item opens or executes

5.4.12. Manual Action Execution

The mobile app can manually execute an action by sending a request to /actions/execute.
Example request:

```
{
  "action": "LIGHT_ON"
}
```

This is useful for:

- Testing device control without using the camera
- Debugging the backend
- Providing manual fallback control
- Demonstrating the system when the Raspberry Pi vision module is not active

5.4.13. Backward Compatibility

The backend includes /event as a backward-compatible alias for /vision-events.
This allows older Pi scripts or simple test clients to keep working while the mobile application uses the newer modular /vision-events endpoint.

5.5. Mobile Application Implementation

The mobile application was implemented using **React Native**, **Expo**, **TypeScript**, **Axios**, and **Zustand**.
The mobile app acts as:

- A smart home control interface
- A backend status monitor
- A mapping configuration panel
- A manual testing tool for actions and vision events

5.5.1. Mobile Dependencies

The mobile app uses:

Dependency	Purpose
Expo	Running and building the React Native app
React Native	Mobile application framework
TypeScript	Type-safe development
Axios	HTTP communication with backend
Zustand	Global state management
React Navigation	Screen and tab navigation
Lucide React Native	UI icons

5.5.2. API Client

The mobile application communicates with the backend through an Axios API client.
The API base URL is read from:
EXPO_PUBLIC_API_URL

If no value is provided, the default URL is:
http://localhost:8000

The API client has a 10-second timeout and uses JSON request headers.

5.5.3. Mobile API Functions

The mobile API client implements the following functions:

Function	Backend Endpoint	Purpose
fetchStatus()	GET /status	Fetch full system, UI, device, and mapping state
fetchVisionEventTypes()	GET /vision-event-types	Fetch supported eye/vision event names
fetchActions()	GET /actions	Fetch supported backend actions
fetchMappings()	GET /mappings	Fetch current mappings and supported lists

updateMappings()	PUT /mappings	Save edited event-to-action mappings
sendVisionEvent()	POST /vision-events	Send test eye/vision events
executeAction()	POST /actions/execute	Manually trigger an action

This means the mobile app works with the same API flow used by the Raspberry Pi. For example, a test button in the mobile app can send a look_right event to /vision-events, just like the Raspberry Pi would after detecting a right gaze.

5.5.4. Mobile Role in Testing

The mobile application was important during integration because it allowed the team to test backend behavior without always running the full camera pipeline.
The following scenarios can be tested from the mobile app:

- Backend connection status
- Manual light on/off commands
- Vision event simulation
- Mapping update behavior
- UI navigation state changes
- Last detected event and last executed action
- Device status returned from backend

This helped validate the backend before and during Raspberry Pi integration.

5.6. Smart Bulb Integration

The working physical device integration in the current implementation is the TP-Link/Tapo smart bulb.
The bulb is controlled over the local network using python-kasa.

5.6.1. Environment-Based Device Configuration

The backend uses environment variables for real device configuration:
SMART_BULB_HOST=<bulb-ip-address>
KASA_USERNAME=<tp-link-account-username>
KASA_PASSWORD=<tp-link-account-password>

If SMART_BULB_HOST is empty, the backend runs in mock mode.
This design allows two different testing modes:

Mode	Description
Mock mode	Backend and mobile app can be tested without physical hardware
Real device mode	LIGHT_ON and LIGHT_OFF are sent to the TP-Link/Tapo bulb

5.6.2. Light Command Flow

The real bulb control flow is:
Action received by backend
↓
Action is normalized
↓
Backend checks if action is a device action
↓
Backend updates internal device status
↓
SmartHomeDeviceController executes command
↓
python-kasa discovers the bulb by host
↓
Bulb is turned on or off
↓
Backend stores last command result

Example:
LIGHT_ON → python-kasa → TP-Link/Tapo bulb turns on

Example:
LIGHT_OFF → python-kasa → TP-Link/Tapo bulb turns off

5.6.3. Plug State

The backend currently supports PLUG_ON and PLUG_OFF as actions and updates the backend plug state.
However, the current device controller does not yet include a real smart plug binding. Therefore, plug actions are part of the backend prototype state and action system, but the successful physical end-to-end demo focuses on smart bulb control.

5.7. End-to-End Integration Flow

The final demo validates the complete path from eye movement to physical smart bulb response.

The end-to-end flow is:

User
↓
IP Camera / Smartphone Camera Stream
↓
Raspberry Pi 5
↓
OpenCV Frame Capture
↓
MediaPipe Face Mesh
↓
GestureDetector
↓
Vision Event Payload
↓
POST /vision-events
↓
FastAPI Backend
↓
Mapping Store
↓
Action Execution
↓
SmartHomeDeviceController
↓
python-kasa
↓
TP-Link/Tapo Smart Bulb
↓
Physical Feedback

5.8. Implementation Decisions

5.8.1. Event-Based Architecture

The most important implementation decision was using an event-based architecture.

The Raspberry Pi detects only user interaction events. It does not decide which smart home command should be executed. This responsibility belongs to the backend.

This separation provides:

- Better modularity
- Easier debugging
- Reconfigurable gesture behavior
- More maintainable code
- Easier future device integration

5.8.2. Local Vision Processing

Vision processing is performed locally on the Raspberry Pi.
This reduces latency and protects user privacy because camera frames do not need to be uploaded to a remote server.

5.8.3. Configurable Mapping

The backend keeps a user-editable mapping table.
This allows the system to change gesture meanings without changing the vision detection code.
Example:
short_blink → SELECT

can be changed to:
short_blink → LIGHT_ON

This makes the system more adaptable for different user needs.

5.8.4. Mock Mode and Real Device Mode

The backend can run in two modes.
In mock mode, the full software stack can be tested without physical hardware.
In real device mode, the backend controls the TP-Link/Tapo bulb through python-kasa.
This was useful because backend, mobile, and Pi modules could be tested independently before the final hardware demo.

5.8.5. Backward-Compatible Endpoint

The backend keeps /event as an alias for /vision-events.
This protects older Pi scripts or test scripts from breaking while the newer modular API structure is used by the mobile application.

5.9. Current Limitations

The current implementation has the following limitations:

- Eye tracking accuracy depends on lighting conditions.
- Camera angle and distance affect gaze detection stability.
- The system currently assumes one active face/user in the camera frame.
- Real hardware control is implemented for the smart bulb only.
- Plug actions exist in backend state but do not yet control a physical smart plug.
- Backend state and mappings are stored in memory, not in a persistent database.
- Authentication and user accounts are not implemented yet.
- The IP camera stream must be reachable over the local network.
- The current implementation focuses on local network operation.
- Calibration may be required for different users, camera positions, and lighting conditions.

5.10. Future Improvements

Future improvements can include:

- Persistent database support for mappings and device states
- Real smart plug integration
- Additional device adapters for other LAN-based smart home devices
- Device discovery from the mobile application
- User-specific calibration profiles
- Improved gaze calibration and sensitivity settings

5.11. Summary

The implemented system successfully demonstrates an accessible, modular, eye-controlled smart home prototype.

The Raspberry Pi processes an IP camera stream locally using OpenCV and MediaPipe Face Mesh. It detects eye movements and blink gestures, then sends standardized vision events to the FastAPI backend. The backend maps these events to configurable actions, updates UI/device state, and executes real smart bulb commands through python-kasa when the TP-Link/Tapo bulb is configured. The mobile application provides control, monitoring, testing, and mapping configuration features.

The successful end-to-end demo proves that the system can convert eye-based interaction into real smart home control. This validates the main goal of the project: providing a low-cost, accessible, and extensible smart home control system for users who may have difficulty using traditional touch-based or voice-based interfaces.

6. Testing & Validation

This section describes how the **IoT Integrated Accessible Smart Home System** was tested and validated. Since the project consists of multiple connected modules, testing was performed in stages: backend testing, Raspberry Pi vision testing, mobile application testing, smart bulb testing, and final end-to-end validation.

The main goal of testing was to verify that the complete system can successfully convert eye-based user interaction into smart home device control.

The validated end-to-end flow is:

Eye Movement / Blink
→ Raspberry Pi Vision Event
→ Backend Event Processing
→ Event-to-Action Mapping
→ Action Execution
→ Smart Bulb Command
→ Physical Device Response

6.1. Testing Strategy

The system was tested using a staged integration strategy.

Instead of testing all components only at the end, each module was tested independently first. After individual modules were verified, they were connected step by step until the full end-to-end demo was completed.

The testing process followed this order:

1. Backend API testing
2. Mobile application testing in mock/backend-connected mode
3. Raspberry Pi vision module testing
4. Smart bulb hardware testing
5. Raspberry Pi → Backend integration testing
6. Mobile → Backend integration testing
7. Final end-to-end demo testing

This approach helped isolate errors and made debugging easier.

6.2. Test Environment

The tests were performed using the following hardware and software environment.

6.2.1. Hardware Environment

Component	Purpose
Raspberry Pi 5	Runs the vision processing module
IP camera / smartphone camera stream	Provides real-time video input

Development computer	Runs backend during development and API testing
Mobile phone	Runs the React Native mobile application
TP-Link/Tapo smart bulb	Real smart device used in the end-to-end demo
Local Wi-Fi network	Connects Raspberry Pi, backend, mobile app, and smart bulb

6.2.2. Software Environment

Component	Technology
Backend	Python, FastAPI, Uvicorn
Device controller	python-kasa
Raspberry Pi vision module	Python, OpenCV, MediaPipe, NumPy
Mobile application	React Native, Expo, TypeScript, Axios, Zustand
Configuration	.env files and environment variables
API testing	Browser, Swagger UI, manual HTTP requests

6.3. Backend API Testing

Backend testing focused on validating that the FastAPI service correctly receives events, updates mappings, executes actions, updates state, and returns consistent responses.

The backend was tested using its HTTP endpoints.

6.3.1. Backend Startup Test

The first test was to verify that the backend starts successfully.

Expected result:

- FastAPI server starts without errors.
- Uvicorn listens on the configured host and port.

- Swagger UI is accessible.
- Root endpoint returns an API running message.

Tested endpoint:
GET /

Expected response example:

```
{
  "message": "Accessible Smart Home Modular API is running"
}
```

Result:
Passed

6.3.2. Health Endpoint Test

The /health endpoint was tested to verify that the backend can report its runtime health, device controller mode, and current UI snapshot.

Tested endpoint:
GET /health

Expected fields:

- success
- device_controller_mode
- ui_state

Expected response example:

```
{
  "success": true,
  "device_controller_mode": "mock",
  "ui_state": {
    "current_screen": "main_menu",
    "selected_index": 0,
    "selected_item": "Light",
    "items": ["Light", "Plug", "Status"]
  }
}
```

Validation criteria:

- success should be true.
- device_controller_mode should be one of mock, python-kasa, or misconfigured.
- UI state should include the current screen, selected index, selected item, and available items.

Result:

Passed

6.3.3. Status Endpoint Test

The /status endpoint was tested to verify that the backend returns all important runtime information.
Tested endpoint:
GET /status

Expected fields:

- success
- system_state
- ui_state
- device_controller_mode
- mappings

The endpoint was expected to return:

- Last received vision event
- Last executed action
- Last device command
- Current light status
- Current plug status
- Current UI screen
- Current selected item
- Current event-to-action mappings

Validation criteria:

- Response must contain both system and UI state.
- Device status must include light and plug.
- Mapping table must include default vision events.
- Response must be valid JSON.

Result:
Passed

6.3.4. Supported Vision Events Test

The /vision-event-types endpoint was tested to verify that the backend exposes the supported vision event names.
Tested endpoint:
GET /vision-event-types

Expected supported events:

look_left
look_right
short_blink
long_blink

Expected response example:

```
{
  "success": true,
  "vision_event_types": [
    "long_blink",
    "look_left",
    "look_right",
    "short_blink"
  ]
}
```

Validation criteria:

- Response must include all four supported vision events.
- No unsupported or outdated event names should be required for the final eye-control flow.

Result:
Passed

6.3.5. Supported Actions Test

The /actions endpoint was tested to verify that the backend exposes all supported actions.
Tested endpoint:
GET /actions

Expected supported actions:
NAV_LEFT
NAV_RIGHT
SELECT
BACK
LIGHT_ON
LIGHT_OFF
PLUG_ON
PLUG_OFF

Validation criteria:

- Response must include UI navigation actions.
- Response must include light control actions.

- Response must include prototype plug actions.

Result:
Passed

6.3.6. Mapping Retrieval Test

The /mappings endpoint was tested to verify that the backend returns the current event-to-action mapping table.
Tested endpoint:
GET /mappings

Expected default mappings:

Vision Event	Default Action
look_left	NAV_LEFT
look_right	NAV_RIGHT
short_blink	SELECT
long_blink	BACK

Validation criteria:

- Mapping table must include all supported vision events.
- Supported vision event list must be returned.
- Supported action list must be returned.

Result:
Passed

6.3.7. Mapping Update Test

The /mappings endpoint was tested with PUT requests to verify that event-to-action mappings can be updated.
Tested endpoint:
PUT /mappings

Example request:

```
{
  "mappings": {
    "short_blink": "LIGHT_ON"
  }
}
```

Expected behavior:

- Backend validates the event name.
- Backend validates the action name.
- Mapping store is updated.
- Updated mapping is returned in the response.

Expected response fields:

- success
- message
- updated
- mappings

Validation criteria:

- Valid mappings should be accepted.
- Invalid event names should be rejected.
- Invalid action names should be rejected.
- Updated mapping should affect later /vision-events requests.

Result:
Passed

6.3.8. Invalid Mapping Test

The backend was tested with unsupported vision events and unsupported actions.
Example invalid event request:

```
{
  "mappings": {
    "eye_up": "NAV_LEFT"
  }
}
```

Expected result:
400 Bad Request

Example invalid action request:

```
{
  "mappings": {
    "look_left": "UNKNOWN_ACTION"
  }
}
```

Expected result:
400 Bad Request

Validation criteria:

- Backend must not accept unsupported event names.
- Backend must not accept unsupported action names.
- Error message should explain which value is unsupported.

Result:
Passed

6.4. Action Execution Testing

Action execution testing focused on verifying that actions change the correct UI state or device state.

6.4.1. Manual Action Execution Test

The /actions/execute endpoint was tested to manually execute actions without requiring camera input.
Tested endpoint:
POST /actions/execute

Example request:

```
{
  "action": "NAV_RIGHT"
}
```

Expected behavior:

- Backend executes NAV_RIGHT.
- UI selected index changes.
- Response returns updated UI state.

Validation criteria:

- Supported actions should execute successfully.
- Unsupported actions should return 400 Bad Request.
- UI and device state should remain consistent after execution.

Result:
Passed

6.4.2. UI Navigation Test

UI navigation was tested using manual action execution.

Test sequence:
Initial state: main_menu, selected item = Light
NAV_RIGHT
NAV_RIGHT
NAV_LEFT

Expected behavior:

Step	Expected Selected Item
Initial	Light
After first NAV_RIGHT	Plug
After second NAV_RIGHT	Status
After NAV_LEFT	Plug

Validation criteria:

- NAV_RIGHT should move selection forward.
- NAV_LEFT should move selection backward.
- Selection should wrap around at list boundaries.

Result:
Passed

6.4.3. Select Action Test

The SELECT action was tested on different screens.
Example sequence:
main_menu → selected item: Light → SELECT

Expected result:
current_screen = light_menu
selected_item = On

Another sequence:
light_menu → selected item: On → SELECT

Expected result:
LIGHT_ON command is executed
light status becomes on

Validation criteria:

- SELECT should open menus from the main menu.
- SELECT should execute device commands inside device menus.
- SELECT should return status information in the status menu.
- SELECT on Back should return to main menu.

Result:
Passed

6.4.4. Back Action Test

The BACK action was tested from different screens.
Example sequence:
main_menu → Light → SELECT → light_menu → BACK

Expected result:
current_screen = main_menu
selected_item = Light

Validation criteria:

- BACK should return to main_menu.
- If already on main_menu, BACK should not break the state.
- UI state should remain valid after back navigation.

Result:
Passed

6.5. Vision Event Processing Testing

Vision event processing tests verified that eye events are correctly mapped to actions.

6.5.1. Vision Event Endpoint Test

The /vision-events endpoint was tested with valid event payloads.
Tested endpoint:
POST /vision-events

Example request:

```
{
  "source": "eye",
  "name": "look_right",
```

```
  "confidence": 1.0,
  "timestamp": 1234567890,
  "metadata": {
    "test": true
  }
}
```

Expected behavior:

- Backend validates look_right.
- Backend stores it as last_vision_event.
- Backend maps it to NAV_RIGHT.
- Backend executes NAV_RIGHT.
- Backend returns updated UI state.

Validation criteria:

- Valid events should be accepted.
- Event should update last_vision_event.
- Mapped action should be executed.
- Response should include mapped_action, action_result, ui_state, and device_status.

Result:
Passed

6.5.2. Default Mapping Behavior Test

The default event mapping was tested.

Vision Event	Expected Action
look_left	NAV_LEFT
look_right	NAV_RIGHT
short_blink	SELECT
long_blink	BACK

Validation criteria:

- Each event should trigger the correct default action.
- UI state should update according to the mapped action.
- Device commands should be executed only when the mapped action leads to a device action.

Result:
Passed

6.5.3. Updated Mapping Behavior Test

After updating mappings, the /vision-events endpoint was tested again.
Example mapping update:

```
{
  "mappings": {
    "short_blink": "LIGHT_ON"
  }
}
```

Example event request:

```
{
  "source": "eye",
  "name": "short_blink",
  "confidence": 1.0,
  "timestamp": 1234567890,
  "metadata": {}
}
```

Expected behavior:
short_blink → LIGHT_ON

Validation criteria:

- Updated mapping should be used immediately.
- No Raspberry Pi code change should be required.
- Backend should execute the newly mapped action.

Result:
Passed

6.5.4. Unsupported Vision Event Test

The backend was tested with unsupported event names.
Example request:

```
{
  "source": "eye",
  "name": "look_up",
  "confidence": 1.0,
  "timestamp": 1234567890,
```

```
  "metadata": {}
}
```

Expected result:
400 Bad Request

Validation criteria:

- Unsupported events should not be accepted.
- Backend should return a clear error message.
- System state should not be corrupted.

Result:
Passed

6.6. Device Control Testing

Device control testing verified both mock mode and real smart bulb control.

6.6.1. Mock Mode Test

The backend was tested without configuring SMART_BULB_HOST.
Expected controller mode:
mock

Expected behavior:

- Backend should still start successfully.
- UI navigation should work.
- Mapping updates should work.
- LIGHT_ON and LIGHT_OFF should update backend state.
- Hardware result should indicate that no real device was applied.

Validation criteria:

- Full software stack should work without physical hardware.
- Device status should update in backend state.
- No crash should occur when smart bulb is not configured.

Result:
Passed

6.6.2. Real Smart Bulb Configuration Test

The backend was then configured with smart bulb environment variables.
Required variables:
SMART_BULB_HOST=<bulb-ip-address>
KASA_USERNAME=<tp-link-account-username>
KASA_PASSWORD=<tp-link-account-password>

Expected controller mode:
python-kasa

Validation criteria:

- Backend should load environment variables.
- Controller should switch from mock mode to python-kasa mode.
- Device discovery should find the bulb at the configured host.
- No import error should occur for python-kasa.

Result:
Passed

6.6.3. Light ON Test

The smart bulb ON command was tested.
Possible trigger methods:
POST /actions/execute with LIGHT_ON

or:
vision event → mapped action → LIGHT_ON

Expected behavior:

- Backend executes LIGHT_ON.
- Internal light state becomes on.
- python-kasa sends turn_on() to the smart bulb.
- The physical smart bulb turns on.
- Response includes hardware result.

Expected backend state:

```
{
  "device_status": {
    "light": "on"
  }
}
```

Result:

Passed

6.6.4. Light OFF Test

The smart bulb OFF command was tested.
Possible trigger methods:
POST /actions/execute with LIGHT_OFF

or:
vision event → mapped action → LIGHT_OFF

Expected behavior:

- Backend executes LIGHT_OFF.
- Internal light state becomes off.
- python-kasa sends turn_off() to the smart bulb.
- The physical smart bulb turns off.
- Response includes hardware result.

Expected backend state:

```
{
  "device_status": {
    "light": "off"
  }
}
```

Result:
Passed

6.6.5. Plug Action Prototype Test

The backend supports PLUG_ON and PLUG_OFF as prototype actions.
Expected behavior:

- PLUG_ON updates backend plug state to on.
- PLUG_OFF updates backend plug state to off.
- Device controller returns a mock/no-binding result for plug commands.

Validation criteria:

- Plug state should update in backend state.
- Backend should not crash when plug actions are executed.
- It should be clear that real plug hardware binding is not implemented yet.

Result:

Passed as backend prototype behavior

6.7. Raspberry Pi Vision Testing

Raspberry Pi testing focused on verifying that the vision module can read the camera stream, detect eye gestures, and publish backend-compatible events.

6.7.1. IP Camera Connection Test

The Pi module was tested with the configured IP camera stream.
Configuration variable:
IP_CAMERA_URL

Expected behavior:

- OpenCV should open the camera stream.
- Frames should be read continuously.
- If the camera cannot be opened, the program should print an error and exit safely.

Validation criteria:

- Camera URL must be reachable from the Raspberry Pi.
- Frame capture should not freeze during normal operation.
- Camera failure should be handled gracefully.

Result:
Passed

6.7.2. Frame Processing Test

Captured frames were tested for correct preprocessing.
Expected processing steps:
Read frame
Resize frame
Convert BGR to RGB
Send RGB frame to MediaPipe Face Mesh

Validation criteria:

- Frame dimensions should match configured width and height.
- Color conversion should be performed before MediaPipe processing.
- Face Mesh should receive valid RGB frames.

Result:
Passed

6.7.3. Face and Eye Landmark Detection Test

MediaPipe Face Mesh was tested for face and eye landmark detection.
Expected behavior:

- One face should be detected when the user is in front of the camera.
- Eye and iris landmarks should be available.
- The system should continue running if no face is detected temporarily.

Validation criteria:

- Face detection should work under normal lighting.
- Eye/iris landmarks should be stable enough for gaze and blink detection.
- The system should not crash when landmarks are unavailable.

Result:
Passed

6.7.4. Gaze Event Test

Left and right gaze events were tested.
Tested events:
look_left
look_right

Expected behavior:

- Looking left for a stable duration should generate look_left.
- Looking right for a stable duration should generate look_right.
- Short accidental eye movements should not immediately trigger events.

Validation criteria:

- Gaze direction must remain stable for the configured duration.
- Events should not fire repeatedly too fast.
- Event name should match backend-supported event names.

Result:
Passed

6.7.5. Blink Event Test

Short and long blink events were tested.
Tested events:
short_blink

long_blink

Expected behavior:

- A short blink within the configured time range should generate short_blink.
- A long blink within the configured time range should generate long_blink.
- Normal involuntary blinks should not create too many false commands.

Validation criteria:

- EAR threshold should detect eye closure.
- Blink duration should separate short and long blinks.
- Event names should match backend-supported event names.

Result:
Passed

6.7.6. Event Payload Test

The generated event payload was validated before backend integration.
Expected payload format:

```
{
 "source": "eye",
 "name": "look_left",
 "confidence": 1.0,
 "timestamp": 1234567890,
 "metadata": {}
}
```

Validation criteria:

- Payload should include source.
- Payload should include name.
- Payload should include timestamp.
- Payload should be compatible with backend VisionEventRequest.

Result:
Passed

6.7.7. Pi to Backend Event Sending Test

The Pi was tested by sending generated events to the backend endpoint.
Backend event URL:
BACKEND_EVENT_URL

Default endpoint:
http://127.0.0.1:8000/vision-events

Expected behavior:

- Pi sends event payload to backend.
- Backend accepts the event.
- Backend executes mapped action.
- Response is printed or logged for debugging.

Validation criteria:

- Backend URL should be reachable from the Raspberry Pi.
- Payload should match backend schema.
- Backend should return a successful response for supported events.

Result:
Passed

6.8. Mobile Application Testing

Mobile testing focused on verifying that the app can communicate with the backend and support control, diagnostics, and mapping configuration.

6.8.1. Backend Connection Test

The mobile app was tested with the configured API URL.
Configuration variable:
EXPO_PUBLIC_API_URL

Expected behavior:

- App can reach the backend API.
- API timeout is handled properly.
- If backend is unavailable, the app shows an error/connection state.

Validation criteria:

- Backend URL must be reachable from the mobile device.
- Mobile device and backend must be on the same network when using a local IP.
- App should not crash when backend is offline.

Result:
Passed

6.8.2. Status Fetch Test

The mobile app was tested by fetching backend status.
API function:
fetchStatus()

Backend endpoint:
GET /status

Expected behavior:

- App receives system state.
- App receives UI state.
- App receives device status.
- App receives mappings.

Validation criteria:

- Response should be parsed successfully.
- App state should update according to backend response.
- Device and event information should be visible in the UI.

Result:
Passed

6.8.3. Vision Event Types Fetch Test

The mobile app was tested by fetching supported vision events.
API function:
fetchVisionEventTypes()

Backend endpoint:
GET /vision-event-types

Expected result:
look_left
look_right
short_blink
long_blink

Validation criteria:

- App should display or use the supported event list.
- Event names should match backend-supported names.

Result:
Passed

6.8.4. Actions Fetch Test

The mobile app was tested by fetching supported actions.
API function:
fetchActions()

Backend endpoint:
GET /actions

Expected actions:
NAV_LEFT
NAV_RIGHT
SELECT
BACK
LIGHT_ON
LIGHT_OFF
PLUG_ON
PLUG_OFF

Validation criteria:

- App should display or use supported action names.
- Action names should match backend-supported names.

Result:
Passed

6.8.5. Mapping Fetch and Update Test

The mobile app was tested by fetching and updating mappings.
API functions:
fetchMappings()
updateMappings()

Backend endpoints:
GET /mappings
PUT /mappings

Expected behavior:

- Current mappings are displayed.

- User can update a mapping.
- Backend accepts valid mapping updates.
- Updated mapping affects future event behavior.

Validation criteria:

- Valid event/action pairs should be accepted.
- Invalid mappings should be rejected by backend.
- UI should reflect updated mappings.

Result:
Passed

6.8.6. Test Vision Event Sending

The mobile app was tested by manually sending vision events to the backend.
API function:
sendVisionEvent()

Backend endpoint:
POST /vision-events

Example test event:

```
{
  "source": "mobile_test",
  "name": "look_right",
  "confidence": 1.0,
  "timestamp": 1234567890,
  "metadata": {
    "origin": "mobile"
  }
}
```

Expected behavior:

- Backend receives the event.
- Backend maps it to the configured action.
- UI/device state changes according to the action.
- App displays updated backend response.

Validation criteria:

- Test events should behave the same way as Raspberry Pi events.
- This confirms that the backend event pipeline is source-independent.

Result:

Passed

6.8.7. Manual Action Execution Test

The mobile app was tested by manually executing actions.
API function:
executeAction()

Backend endpoint:
POST /actions/execute

Example request:

```
{
  "action": "LIGHT_ON"
}
```

Expected behavior:

- Backend executes the requested action.
- Device state is updated.
- If smart bulb is configured, real bulb command is sent.
- App receives action result.

Validation criteria:

- Manual action execution should work without the camera.
- This provides an accessibility fallback and debugging method.

Result:
Passed

6.9. End-to-End Validation

End-to-end validation tested the complete system as one integrated product.

6.9.1. End-to-End Test Setup

Before running the final demo, the following conditions were checked:

- Backend server was running.
- Raspberry Pi was connected to the same local network.
- IP camera stream was reachable from the Raspberry Pi.
- Mobile app was configured with the correct backend URL.

- Smart bulb was connected to the same local network.
- SMART_BULB_HOST was configured in backend .env.
- KASA_USERNAME and KASA_PASSWORD were configured when required.
- Backend controller mode returned python-kasa.
- /status endpoint returned valid state.

Result:
Passed

6.9.2. Full Demo Flow Test

The final demo tested the following complete flow:
User eye movement
→ IP camera frame capture
→ Raspberry Pi MediaPipe processing
→ GestureDetector event generation
→ POST /vision-events
→ Backend mapping lookup
→ Backend action execution
→ SmartHomeDeviceController
→ python-kasa
→ TP-Link/Tapo smart bulb

Expected behavior:

- User can navigate using eye direction.
- User can select using blink gesture.
- Backend processes the event correctly.
- Smart bulb command is executed.
- Physical bulb state changes.

Result:
Passed

6.9.3. Example Successful Scenario

The following scenario was successfully validated:
1. User looks right.
2. Raspberry Pi detects stable right gaze.
3. Pi generates look_right event.
4. Backend receives the event through /vision-events.
5. Backend maps look_right to NAV_RIGHT.
6. UI selected item changes.

7. User performs short blink.
8. Pi generates short_blink event.
9. Backend maps short_blink to SELECT.
10. Backend opens selected menu or executes selected action.
11. User navigates to light control.
12. User selects light ON.
13. Backend executes LIGHT_ON.
14. SmartHomeDeviceController sends the command using python-kasa.
15. TP-Link/Tapo smart bulb turns on.

Validation result:
Successful

6.9.4. Physical Device Response Validation

Physical smart bulb response was validated visually.
Tested commands:

Command	Expected Physical Result	Result
LIGHT_ON	Smart bulb turns on	Passed
LIGHT_OFF	Smart bulb turns off	Passed

This confirms that the system can control a real smart home device, not only update software state.

6.10. Error Handling and Edge Case Validation

The system was also tested against common error cases.

6.10.1. Backend Offline Case

Scenario:
Mobile app or Raspberry Pi tries to send a request while backend is not running.

Expected behavior:

- Request fails gracefully.
- App/Pi logs or displays an error.
- System does not crash.

Result:

Passed

6.10.2. Camera Unavailable Case

Scenario:
IP camera stream is unavailable or wrong camera URL is configured.

Expected behavior:

- Pi cannot open the camera.
- Error is printed.
- Program exits safely instead of crashing unexpectedly.

Result:
Passed

6.10.3. Unsupported Vision Event Case

Scenario:
Unsupported event name is sent to backend.

Example:

```
{
  "source": "eye",
  "name": "look_up",
  "confidence": 1.0,
  "timestamp": 1234567890,
  "metadata": {}
}
```

Expected behavior:
400 Bad Request

Result:
Passed

6.10.4. Unsupported Action Case

Scenario:
Unsupported action is sent to /actions/execute.

Example:

```
{
  "action": "UNKNOWN_ACTION"
}
```

Expected behavior:
400 Bad Request

Result:
Passed

6.10.5. Smart Bulb Not Configured Case

Scenario:
SMART_BULB_HOST is not configured.

Expected behavior:

- Backend controller mode becomes mock.
- Software action pipeline still works.
- Device status updates in backend state.
- Hardware result indicates that real device command was not applied.

Result:
Passed

6.10.6. Smart Bulb Connection Failure Case

Scenario:
SMART_BULB_HOST is configured but the device cannot be reached.

Expected behavior:

- Backend attempts device discovery.
- If discovery fails, backend returns an error.
- Error is reported as device execution failure.
- Failure does not corrupt mapping or UI state.

Result:
Handled by backend error response

6.11. Validation Summary

The testing process confirmed that the implemented system satisfies the main project goal: enabling smart home control through eye-based interaction.

The following major points were validated:

- Raspberry Pi can process an IP camera stream.
- MediaPipe Face Mesh can be used for eye landmark detection.
- The system can detect look_left, look_right, short_blink, and long_blink.
- Vision events can be sent from Raspberry Pi to backend.
- Backend can map vision events to actions.
- Mappings can be updated from the mobile application.
- Backend can manage UI navigation state.
- Backend can manage device state.
- Mobile app can monitor and control backend behavior.
- LIGHT_ON and LIGHT_OFF can control a real TP-Link/Tapo smart bulb through python-kasa.
- The final end-to-end demo was completed successfully.

6.12. Current Testing Limitations

Although the main end-to-end demo was successful, some limitations remain:

- Tests were primarily performed in a controlled local network environment.
- Eye tracking accuracy was tested mainly under normal lighting conditions.
- Long-term continuous runtime testing has not yet been performed.
- Multi-user camera scenarios were not tested.
- Real plug hardware was not integrated yet.
- Plug actions were validated only as backend prototype state changes.
- Automated unit and integration tests are not yet included.
- Persistent database behavior was not tested because the current backend stores state in memory.
- Authentication and security testing were not performed because user accounts are not implemented yet.

6.13. Future Testing Improvements

Future testing work can include:

- Automated backend unit tests
- Automated API integration tests
- Mobile UI tests
- Long-duration Raspberry Pi runtime tests

- Camera distance tests
- Lighting condition tests
- Low-light and backlight tests
- Multiple user/no-face edge case tests
- Smart bulb reconnection tests
- Real smart plug integration tests
- Latency measurement from gesture to device response
- User experience testing with different users
- Accessibility testing with target user groups
- Persistent storage testing after database integration
- Security testing after authentication is added

6.14. Conclusion

The testing and validation process showed that the system works as a complete accessible smart home prototype. Individual modules were tested separately and then integrated step by step. The final end-to-end demo successfully validated the complete flow from eye movement and blink detection to backend action execution and real TP-Link/Tapo smart bulb control.

The successful validation proves that the project can provide a practical hands-free interaction method for smart home control. It also confirms that the modular event-to-action architecture is suitable for future expansion with more devices, improved calibration, persistent storage, and additional accessibility features.

7. Conclusion & Future Work

7.1. Project Summary

The IoT Integrated Accessible Smart Home System was designed and implemented as a three-tier architecture that enables hands-free control of smart home devices through eye movements, hand gestures, and face-based authentication. The system consists of a Raspberry Pi 5 vision processing pipeline, a FastAPI backend server, and a React Native (Expo) mobile application, all communicating over a local area network without cloud dependency.

The primary goal of providing an accessible, non-contact control interface for individuals with motor disabilities has been achieved. The system successfully demonstrates the complete end-to-end flow:

User Gesture → Pi Vision Detection → Backend Event Processing → Action Mapping → Device Command → Physical Device Response

Users can navigate menus, toggle smart devices (TP-Link Tapo L530E smart bulb, P100 smart plug), and receive spoken feedback — all without touching any physical interface.

7.2. Key Achievements

The following key achievements were accomplished during the project:

- **Real-time multi-modal gesture recognition:** The Raspberry Pi pipeline processes eye tracking (via MediaPipe Face Mesh) and hand gesture detection (via MediaPipe Hands) simultaneously using a multi-threaded architecture that achieves 10+ FPS on Raspberry Pi 5 hardware.
- **Robust eye tracking with adaptive calibration:** The Eye Aspect Ratio (EAR) baseline dynamically adapts to each user's eye characteristics, enabling reliable detection of short blinks (0.08–0.45s), long blinks (0.45–1.8s), and sustained gaze direction (left/right) across different users without manual calibration.
- **Comprehensive hand gesture vocabulary:** Six distinct hand gestures are recognized — swipe left, swipe right, pinch, open palm hold, thumbs up, and thumbs down — with multiple anti-jitter mechanisms (cooldown, lock-until-release, post-swipe pinch blocking) to prevent false activations.
- **Face-based access control:** An OpenCV LBPH face recognizer provides biometric authentication with a consecutive-vote mechanism (requiring 2 agreeing frames) to prevent single-frame false positives, ensuring only authorized users can control smart home devices.
- **Flexible event-to-action mapping system:** Users can customize which gesture triggers which action through the mobile application. The mapping system supports both eye and hand events and allows runtime reconfiguration without restarting any component.
- **Dual-mode mobile application:** The React Native application offers both a standard smart home dashboard and an accessible mode with enlarged buttons, high-contrast visuals, and Turkish text-to-speech feedback (via expo-speech) for every device action.
- **Cloud-free local architecture:** All processing — vision inference, event routing, device control — occurs entirely on the local network. No internet connection or cloud service is required for operation, ensuring privacy, low latency, and reliability.
- **Mock/real dual-mode device controller:** The backend operates in mock mode when no physical smart devices are configured, enabling full system development and testing without hardware. When TP-Link Tapo device credentials are provided via environment variables, it seamlessly switches to real device control.

7.3. Current Limitations

While the system meets its core objectives, the following limitations have been identified:

- **Single-user vision processing:** The current pipeline processes only one face and one hand at a time. Multi-user concurrent control is not supported.
- **Fixed camera orientation:** The vision pipeline assumes a specific camera orientation (landscape via DroidCam with 90° rotation compensation). Different camera setups may require recalibration of gesture parameters.
- **Limited device ecosystem:** The current implementation supports only TP-Link Tapo L530E (smart bulb) and P100 (smart plug). Integration with other smart home protocols (Zigbee, Z-Wave, Matter) is not yet implemented.
- **No persistent state:** The backend stores all state in memory. A server restart resets device status, mappings, and UI state to defaults. There is no database or persistent storage layer.
- **Network dependency between components:** All three tiers must be on the same local network. There is no remote access or VPN support for controlling devices from outside the home.

7.4. Future Work

The following improvements and extensions are identified for future development:

7.4.1. Enhanced Gesture Recognition

- Integration of additional gesture types such as head tilt, mouth opening, and facial expressions using MediaPipe Face Mesh's full landmark set.
- Machine learning-based gesture classification (e.g., a lightweight CNN or LSTM) to improve recognition accuracy and support user-specific gesture profiles.
- Support for hybrid control mode where both eye and hand inputs are processed simultaneously with conflict resolution.

7.4.2. Expanded Device Ecosystem

- Integration with the Matter smart home standard to support a wider range of devices from different manufacturers.
- Support for additional device types including smart curtains/blinds, door locks, thermostats, and security cameras.
- Implementation of device scenes and automation rules (e.g., "bedtime" scene that turns off all lights and locks doors).

7.4.3. Improved Accessibility Features

- Voice command integration as an additional input modality alongside eye and hand gestures.
- Multi-language TTS support beyond Turkish, with automatic language detection.
- Screen reader compatibility and WCAG-compliant UI elements in the mobile application.
- User-specific gesture sensitivity profiles that can be saved and loaded per user.

7.4.4. System Infrastructure

- Persistent storage using SQLite or PostgreSQL for device state, user preferences, event history, and gesture mappings.
- WebSocket-based real-time communication between backend and mobile app to replace HTTP polling.
- Secure remote access via VPN or end-to-end encrypted tunnel for controlling devices outside the local network.
- Containerized deployment (Docker) for easier installation and reproducibility.

7.4.5. User Experience

- Onboarding wizard in the mobile app to guide first-time users through gesture calibration and device setup.
- Gesture training mode where users can practice gestures with real-time visual feedback before using the system.
- Dashboard widgets showing gesture recognition statistics, device usage history, and system health over time.
- Tablet-optimized layout for wall-mounted control panels.

7.5. Closing Statement

This project demonstrates that computer vision-based interaction can serve as a practical and effective interface for smart home control, particularly for users with motor disabilities. By combining eye tracking, hand gesture recognition, and face-based authentication into a unified, cloud-free system, the project provides a foundation that can be extended into a comprehensive assistive technology platform. The modular three-tier architecture ensures that each component — vision processing, backend logic, and user interface — can be independently improved and scaled as the technology matures.